WYDZIAŁ
MATEMATYKI
I FIZYKI STOSOWANEJ
POLITECHNIKI RZESZOWSKIEJ

Projekt z przedmiotu Aplikacje Bazodanowe

Aplikacja obsługi Portu Morskiego

Jakub Hutman

Krzysztof Koza

L3

Rzeszów, 2026

1. Wstęp. 3
1.1. Cele Projektu 3
2. Tworzenie struktury bazy danych 4
2.1. Definicja i projekt bazy danych 4
2.2. Definicja tabel 4
2.3. Powiązania między tabelami 13
2.4. Wprowadzenie danych do tabeli 18
2.5 Wygląd bazy 22
3. Implementacja pakietów 23
3.1. Pakiet operacje doku 23
3.2. Pakiet Logistyka 41
4. Obsługa błędów 55
5. Strony APEX. 58
6. Eksport i import. 61

1. Wstęp.

1.1. Cele Projektu

Celem realizowanego projektu było zaprojektowanie oraz implementacja aplikacji bazodanowej wspomagającej zarządzanie **hubem transportu morskiego**. System został opracowany jako narzędzie przeznaczone dla pracowników portu morskiego, umożliwiające sprawne zarządzanie danymi operacyjnymi związanymi z obsługą statków, ładunków, infrastruktury portowej oraz zasobów technicznych i ludzkich. Projekt łączy w sobie elementy relacyjnej bazy danych, logiki biznesowej zaimplementowanej w języku PL/SQL oraz aplikacji webowej stworzonej w środowisku Oracle APEX.

Hub transportowy stanowi kluczowy element infrastruktury logistycznej, w którym realizowane są złożone procesy obejmujące planowanie i realizację przeładunków, koordynację pracy doków i dźwigów, kontrolę dostępności obsady oraz zarządzanie harmonogramami operacyjnymi. Efektywne funkcjonowanie takiego obiektu wymaga spójnego systemu informatycznego umożliwiającego centralne gromadzenie danych oraz automatyzację wybranych procesów decyzyjnych. Odpowiedzią na te potrzeby jest zaprojektowana w ramach projektu aplikacja bazodanowa.

Pierwszym etapem realizacji projektu było zaprojektowanie relacyjnej bazy danych w środowisku Oracle Database. Struktura bazy została opracowana zgodnie z zasadami normalizacji, co pozwoliło na eliminację redundancji danych, zachowanie integralności referencyjnej oraz zwiększenie czytelności i elastyczności modelu danych. Baza obejmuje tabele opisujące m.in. statki, ładunki, typy ładunków, doki, dźwigi, harmonogramy przeładunkowe, warunki ubezpieczenia oraz obsadę pracowniczą. Relacje pomiędzy tabelami zostały zdefiniowane za pomocą kluczy głównych i obcych, zapewniając logiczną spójność danych.

Kolejnym etapem projektu była implementacja funkcji oraz procedur składowanych w języku PL/SQL. Ich zadaniem jest realizacja kluczowych operacji biznesowych, takich jak weryfikacja dostępności infrastruktury portowej, sprawdzanie gotowości zasobów technicznych i personalnych, a także wyliczanie parametrów operacyjnych, w tym przybliżonego czasu realizacji przeładunku statku. Zastosowanie logiki po stronie bazy danych pozwala na centralizację reguł biznesowych oraz ograniczenie błędów wynikających z niewłaściwego wprowadzania danych przez użytkowników.

Integralną częścią projektu jest aplikacja webowa wykonana w technologii **Oracle APEX**, która stanowi warstwę prezentacji systemu. Aplikacja umożliwia użytkownikom intuicyjne zarządzanie danymi poprzez formularze, raporty oraz interaktywne widoki, bez konieczności bezpośredniej pracy z językiem SQL. Oracle APEX zapewnia szybkie tworzenie aplikacji opartych na bazie danych Oracle oraz łatwą integrację z procedurami i funkcjami PL/SQL, co znacząco usprawnia proces wdrażania systemu.

Rezultatem projektu jest kompletna aplikacja bazodanowa wspierająca zarządzanie operacjami w hubie transportu morskiego. System ten stanowi praktyczny przykład zastosowania wiedzy z zakresu projektowania relacyjnych baz danych, języka SQL i PL/SQL oraz tworzenia aplikacji bazodanowych w środowisku Oracle APEX. Zaprojektowane rozwiązanie może stanowić podstawę do dalszej rozbudowy systemu oraz jego adaptacji do rzeczywistych warunków funkcjonowania portu morskiego.

2. Tworzenie struktury bazy danych

2.1. Definicja i projekt bazy danych

Struktura bazy danych została zaprojektowana w środowisku Oracle Database zgodnie z zasadami relacyjnego modelu danych. Każda tabela reprezentuje odrębny obiekt występujący w procesach logistycznych hubu transportu morskiego, co pozwala na czytelne i spójne odwzorowanie rzeczywistych zależności.

2.2. Definicja tabel

- Tabela: CIEZAROWKA

Rejestr ciężarówek obsługujących transport lądowy (do/od terminala).

```
CREATE TABLE ciezarowka (
    ciezarowkaid       INTEGER NOT NULL,
    numerrejestracyjny VARCHAR2(20),
    typ                VARCHAR2(100),
    nosnosc            NUMBER(10, 2),
    status             VARCHAR2(50)
)
```

- Tabela: DOK

Podstawowa ewidencja doków przeładunkowych.

```
CREATE TABLE dok (
    dokid        INTEGER NOT NULL,
    nazwa        VARCHAR2(255),
    maks_nosnosc NUMBER(10, 2)
)
```

- Tabela: DOK_NOSNOSC

Rozszerzenie informacji o doku (parametry nośności/pojemności operacyjnej).

```
CREATE TABLE dok_nosnosc (
    dokid                    INTEGER NOT NULL,
    maksymalnanosnosc         NUMBER(10, 2),
    liczbamiejsccumowniczych INTEGER
)
```

- Tabela: DOK_STAN

Aktualny stan doku (np. dostępny, zajęty, serwis).

```
CREATE TABLE dok_stan (
    dokid INTEGER NOT NULL,
    stan  VARCHAR2(50) NOT NULL
)
```

- Tabela: DZWIG

Ewidencja dźwigów portowych przypisanych do stref.

```
CREATE TABLE dzwig (
    dzwigid            INTEGER NOT NULL,
    strefaid           INTEGER,
    maksymalnanosnosc  NUMBER(10, 2),
    typ                VARCHAR2(100),
    status             VARCHAR2(50)
)
```

- Tabela: HARMONOGRAMDOKOW

Harmonogram rezerwacji doków dla operacji (czas + statek + dok).

```
CREATE TABLE harmonogramdokow (
    harmonogramdokowid INTEGER NOT NULL,
    dokid              INTEGER NOT NULL,
    datarozpoczecia    DATE,
    datazakonczenia    DATE,
    statekid           INTEGER NOT NULL
)
```

- Tabela: HARMONOGRAMDOKOW_STAN

Stan pozycji harmonogramu (np. zaplanowany / w trakcie / zakończony).

```
CREATE TABLE harmonogramdokow_stan (
    harmonogramdokowid INTEGER NOT NULL,
    stan               VARCHAR2(50) NOT NULL
)
```

- Tabela: HARMONOGRAMDOKOW_TYP

Typ operacji przypisany do pozycji harmonogramu (np. załadunek/rozładunek).

```
CREATE TABLE harmonogramdokow_typ (
    harmonogramdokowid INTEGER NOT NULL,
    typoperacji        VARCHAR2(50) NOT NULL
)
```

- Tabela: HARMONOGRAMPRACOWNIKA

Ewidencja przypisania pracownika do harmonogramu pracy (grafik).

```
CREATE TABLE harmonogrampracownika (
    harmonogrampracownikaid INTEGER NOT NULL,
    pracownikid             INTEGER NOT NULL,
    dataod                  DATE,
    datado                  DATE,
    zmianapracownika        VARCHAR2(50)
)
```

- Tabela: KLIENT

Klienci (nadawcy/odbiorcy) przesyłek obsługiwanych w hubie.

```
CREATE TABLE klient (
    klientid      INTEGER NOT NULL,
    nazwa         VARCHAR2(255),
    adres         VARCHAR2(255),
    numerkontaktu VARCHAR2(20)
)
```

- Tabela: KONTENER

Kontenery wykorzystywane do składowania i transportu przesyłek.

```
CREATE TABLE kontener (
    kontenerid   INTEGER NOT NULL,
    typ          VARCHAR2(50),
    nosnosc      NUMBER(10, 2),
    typkontenera VARCHAR2(100),
    wymiary      VARCHAR2(255),
    waga         NUMBER(10, 2)
)
```

- Tabela: KONTENER_LOKALIZACJA

Bieżąca lokalizacja kontenera (np. strefa/magazyn/dok).

```
CREATE TABLE kontener_lokalizacja (
    kontenerid  INTEGER NOT NULL,
    lokalizacja VARCHAR2(255)
)
```

- Tabela: KONTENER_STATUS

Status kontenera (np. wolny, w użyciu, w drodze, serwis).

```
CREATE TABLE kontener_status (
    kontenerid INTEGER NOT NULL,
    status     VARCHAR2(50) NOT NULL
)
```

- Tabela: MAGAZYN

Magazyny w hubie (z pojemnością, zajętością i statusem).

```
CREATE TABLE magazyn (
    magazynid           INTEGER NOT NULL,
    strefaid            INTEGER NOT NULL,
    typmagazynu         VARCHAR2(50),
    maksymalnapojemnosc NUMBER(10, 2),
    zajetaprzestrzen    NUMBER(10, 2),
    status              VARCHAR2(50)
)
```

- Tabela: PRACOWNIK

Pracownicy portu/hubu (dane personalne + status + strefa robocza).

```
CREATE TABLE pracownik (
    pracownikid      INTEGER NOT NULL,
    imie             VARCHAR2(100),
    nazwisko         VARCHAR2(100),
    stanowisko       VARCHAR2(100),
    numerkontaktu    VARCHAR2(20),
    status           VARCHAR2(50),
    strefa_strefaid  INTEGER NOT NULL,
    harmonogram      VARCHAR2(255)
)
```

- Tabela: PRACOWNIK_STREFA

Przypisanie pracowników do stref

```
CREATE TABLE pracownik_strefa (
    pracownikid INTEGER NOT NULL,
    strefaid    INTEGER NOT NULL
)
```

- Tabela: PRZELADUNEK

Rejestr operacji przeładunkowych (statek + dok/harmonogram + ciężarówka).

```
CREATE TABLE przeladunek (
    przeladunekid                INTEGER NOT NULL,
    statekid                     INTEGER NOT NULL,
    ciezarowkaid                 INTEGER NOT NULL,
    status                       VARCHAR2(50),
    harmonogramdokowid           INTEGER NOT NULL,
    przeladunek_czas_przeladunekid INTEGER NOT NULL
)
```

- Tabela: PRZELADUNEK_CZAS

Czas trwania przeładunku

```
CREATE TABLE przeladunek_czas (
    przeladunekid INTEGER NOT NULL,
    czastrwania   NUMBER(10, 2)
)
```

- Tabela: PRZELADUNEK_PRZESYLKA

Połączenie przeładunków z przesyłkami

```
CREATE TABLE przeladunek_przesylka (
    id            INTEGER NOT NULL,
    przeladunekid INTEGER NOT NULL,
    przesylkaid   INTEGER NOT NULL
)
```

- Tabela: PRZESYLKA

Główna tabela przesyłek

```
CREATE TABLE przesylka (
    przesylkaid              INTEGER NOT NULL,
    typprzesylkiid            INTEGER NOT NULL,
    waga                      NUMBER(10, 2),
    wymiary                   VARCHAR2(255),
    klientid                  INTEGER NOT NULL,
    magazynid                 INTEGER NOT NULL,
    dataprzyjecia             DATE,
    datawydania               DATE,
    status                    VARCHAR2(50),
    ubezpieczenieid           INTEGER NOT NULL,
    kontener_kontenerid       INTEGER NOT NULL,
    przeladunek_przeladunekid INTEGER NOT NULL
)
```

- Tabela: PRZESYLKA_KONTENER

Połączenie przesyłek z kontenerami

```
CREATE TABLE przesylka_kontener (
    id          INTEGER NOT NULL,
    przesylkaid INTEGER NOT NULL,
    kontenerid  INTEGER NOT NULL,
    ilosc       NUMBER(10, 2),
    datadodania DATE
)
```

- Tabela: STATEK

Statki obsługiwane przez hub (parametry i terminy).

```
CREATE TABLE statek (
    statekid        INTEGER NOT NULL,
    nazwa           VARCHAR2(255),
    typ             VARCHAR2(100),
    nosnosc         NUMBER(10, 2),
    flaga           VARCHAR2(50),
    dataprzybycia   DATE,
    dataodplyniecia DATE,
    status          VARCHAR2(50)
)
```

- Tabela: STREFA

Strefy operacyjne w porcie (np. terminal, magazyny, obszary serwisowe).

```
CREATE TABLE strefa (
    strefaid INTEGER NOT NULL,
    nazwa    VARCHAR2(255),
    opis     unknown
)
```

- Tabela: TYP_UBEZPIECZENIA

Poziomy/rodzaje ubezpieczenia

```
CREATE TABLE typ_ubezpieczenia (
    typubezpieczenia            VARCHAR2(50) NOT NULL,
    kwota                       NUMBER(10, 2),
    ubezpieczenie_ubezpieczenieid INTEGER NOT NULL
)
```

- Tabela: TYPPRZESYLKI

Słownik typów przesyłek (np. standard, ADR, chłodnicza).

```
CREATE TABLE typprzesylki (
    typprzesylkiid INTEGER NOT NULL,
    nazwa          VARCHAR2(255),
    wymagania      unknown
)
```

- Tabela: UBEZPIECZENIE

Polisy/okresy ubezpieczenia.

```
CREATE TABLE ubezpieczenie (
    ubezpieczenieid INTEGER NOT NULL,
    datapoczatku    DATE,
    datakonca       DATE
)
```

- Tabela: WARUNKI_UBEZPIECZENIA

Warunki przypisane do typu ubezpieczenia

```
CREATE TABLE warunki_ubezpieczenia (
    warunki         VARCHAR2(50) NOT NULL,
    typubezpieczenia VARCHAR2(50) NOT NULL
)
```

2.3. Powiązania między tabelami

Aby zapewnić logiczną spójność danych, zastosowano relacje pomiędzy tabelami z wykorzystaniem kluczy obcych (FOREIGN KEY). Relacje te wymuszają istnienie rekordów nadrzędnych i zapobiegają wprowadzaniu niespójnych danych w systemie.

- Powiązanie tabeli DOK_NOSNOSC z tabelą DOK

```
ALTER TABLE dok_nosnosc
ADD CONSTRAINT dok_nosnosc_dok_fk
FOREIGN KEY (dokid) REFERENCES dok(dokid);
```

- Powiązanie tabeli DOK_STAN z tabelą DOK

```
ALTER TABLE dok_stan
ADD CONSTRAINT dok_stan_dok_fk
FOREIGN KEY (dokid) REFERENCES dok(dokid);
```

- Powiązanie tabeli DZWIG z tabelą STREFA

```
ALTER TABLE dzwig
ADD CONSTRAINT dzwig_strefa_fk
FOREIGN KEY (strefaid) REFERENCES strefa(strefaid);
```

- Powiązanie tabeli HARMONOGRAMDOKOW z tabelą DOK

```
ALTER TABLE harmonogramdokow
ADD CONSTRAINT harmonogramdokow_dok_fk
FOREIGN KEY (dokid) REFERENCES dok(dokid);
```

- Powiązanie tabeli HARMONOGRAMDOKOW z tabelą STATEK

```
ALTER TABLE harmonogramdokow
ADD CONSTRAINT harmonogramdokow_statek_fk
FOREIGN KEY (statekid) REFERENCES statek(statekid);
```

- Powiązanie tabeli HARMONOGRAMDOKOW_TYP z tabelą HARMONOGRAMDOKOW

```
ALTER TABLE harmonogramdokow_typ
ADD CONSTRAINT hdt_hdok_fk
FOREIGN KEY (harmonogramdokowid) REFERENCES harmonogramdokow(harmonogramdokowid);
```

- Powiązanie tabeli HARMONOGRAMDOKOW_STAN z tabelą HARMONOGRAMDOKOW

```
ALTER TABLE harmonogramdokow_stan
ADD CONSTRAINT hds_hdok_fk
FOREIGN KEY (harmonogramdokowid) REFERENCES harmonogramdokow(harmonogramdokowid);
```

- Powiązanie tabeli HARMONOGRAMPRACOWNIKA z tabelą PRACOWNIK

```
ALTER TABLE harmonogrampracownika
ADD CONSTRAINT hp_prac_fk
FOREIGN KEY (pracownikid) REFERENCES pracownik(pracownikid);
```

- Powiązanie tabeli KONTENER_LOKALIZACJA z tabelą KONTENER

```
ALTER TABLE kontener_lokalizacja
ADD CONSTRAINT kl_kont_fk
FOREIGN KEY (kontenerid) REFERENCES kontener(kontenerid);
```

- Powiązanie tabeli KONTENER_STATUS z tabelą KONTENER

```
ALTER TABLE kontener_status
ADD CONSTRAINT kontener_status_kontener_fk
FOREIGN KEY (kontenerid) REFERENCES kontener(kontenerid);
```

- Powiązanie tabeli MAGAZYN z tabelą STREFA

```
ALTER TABLE magazyn
ADD CONSTRAINT magazyn_strefa_fk
FOREIGN KEY (strefaid) REFERENCES strefa(strefaid);
```

- Powiązanie tabeli PRACOWNIK_STREFA z tabelą PRACOWNIK

```
ALTER TABLE pracownik_strefa
ADD CONSTRAINT pracownik_strefa_pracownik_fk
FOREIGN KEY (pracownikid) REFERENCES pracownik(pracownikid);
```

- Powiązanie tabeli PRACOWNIK_STREFA z tabelą STREFA

```
ALTER TABLE pracownik_strefa
ADD CONSTRAINT pracownik_strefa_strefa_fk
FOREIGN KEY (strefaid) REFERENCES strefa(strefaid);
```

- Powiązanie tabeli PRZELADUNEK z tabelą CIEZAROWKA

```
ALTER TABLE przeladunek
ADD CONSTRAINT przeladunek_ciezarowka_fk
FOREIGN KEY (ciezarowkaid) REFERENCES ciezarowka(ciezarowkaid);
```

- Powiązanie tabeli PRZELADUNEK z tabelą HARMONOGRAMDOKOW

```
ALTER TABLE przeladunek
ADD CONSTRAINT przel_hdok_fk
FOREIGN KEY (harmonogramdokowid) REFERENCES harmonogramdokow(harmonogramdokowid);
```

- Powiązanie tabeli PRZELADUNEK z tabelą PRZELADUNEK_CZAS

```
ALTER TABLE przeladunek
ADD CONSTRAINT przz_czas_fk
FOREIGN KEY (przeladunek_czas_przeladunekid) REFERENCES przeladunek_czas(przeladunekid);
```

- Powiązanie tabeli PRZELADUNEK z tabelą STATEK

```
ALTER TABLE przeladunek
ADD CONSTRAINT przeladunek_statek_fk
FOREIGN KEY (statekid) REFERENCES statek(statekid);
```

- Powiązanie tabeli PRZELADUNEK_PRZESYLKA z tabelą PRZELADUNEK

```
ALTER TABLE przeladunek_przesylka
ADD CONSTRAINT pp_przel_fk
FOREIGN KEY (przeladunekid) REFERENCES przeladunek(przeladunekid);
```

- Powiązanie tabeli PRZELADUNEK_PRZESYLKA z tabelą PRZESYLKA

```
ALTER TABLE przeladunek_przesylka
ADD CONSTRAINT pp_przes_fk
FOREIGN KEY (przesylkaid) REFERENCES przesylka(przesylkaid);
```

- Powiązanie tabeli PRZESYLKA z tabelą KLIENT

```
ALTER TABLE przesylka
ADD CONSTRAINT przesylka_klient_fk
FOREIGN KEY (klientid) REFERENCES klient(klientid);
```

- Powiązanie tabeli PRZESYLKA z tabelą MAGAZYN

```
ALTER TABLE przesylka
ADD CONSTRAINT przesylka_magazyn_fk
FOREIGN KEY (magazynid) REFERENCES magazyn(magazynid);
```

- Powiązanie tabeli PRZESYLKA z tabelą TYPPRZESYLKI

```
ALTER TABLE przesylka
ADD CONSTRAINT przesylka_typprzesylki_fk
FOREIGN KEY (typprzesylkiid) REFERENCES typprzesylki(typprzesylkiid);
```

- Powiązanie tabeli PRZESYLKA z tabelą UBEZPIECZENIE

```
ALTER TABLE przesylka
ADD CONSTRAINT przesylka_ubezpieczenie_fk
FOREIGN KEY (ubezpieczenieid) REFERENCES ubezpieczenie(ubezpieczenieid);
```

- Powiązanie tabeli PRZESYLKA_KONTENER z tabelą KONTENER

```
ALTER TABLE przesylka_kontener
ADD CONSTRAINT przesylka_kontener_kontener_fk
FOREIGN KEY (kontenerid) REFERENCES kontener(kontenerid);
```

- Powiązanie tabeli TYP_UBEZPIECZENIA z tabelą UBEZPIECZENIE

```
ALTER TABLE typ_ubezpieczenia
ADD CONSTRAINT tu_ubez_fk
FOREIGN KEY (ubezpieczenie_ubezpieczenieid) REFERENCES ubezpieczenie(ubezpieczenieid);
```

- Powiązanie tabeli WARUNKI_UBEZPIECZENIA z tabelą TYP_UBEZPIECZENIA

```
ALTER TABLE warunki_ubezpieczenia
ADD CONSTRAINT wu_typ_fk
FOREIGN KEY (typubezpieczenia) REFERENCES typ_ubezpieczenia(typubezpieczenia);
```

2.4. Wprowadzenie danych do tabeli

```
INSERT INTO strefa (strefaid, nazwa, opis) VALUES (1, 'Strefa 1', 'Opis strefy 1');
INSERT INTO strefa (strefaid, nazwa, opis) VALUES (2, 'Strefa 2', 'Opis strefy 2');
INSERT INTO strefa (strefaid, nazwa, opis) VALUES (3, 'Strefa 3', 'Opis strefy 3');
INSERT INTO strefa (strefaid, nazwa, opis) VALUES (4, 'Strefa 4', 'Opis strefy 4');
INSERT INTO strefa (strefaid, nazwa, opis) VALUES (5, 'Strefa 5', 'Opis strefy 5');
INSERT INTO strefa (strefaid, nazwa, opis) VALUES (6, 'Strefa 6', 'Opis strefy 6');
INSERT INTO strefa (strefaid, nazwa, opis) VALUES (7, 'Strefa 7', 'Opis strefy 7');
INSERT INTO strefa (strefaid, nazwa, opis) VALUES (8, 'Strefa 8', 'Opis strefy 8');
INSERT INTO strefa (strefaid, nazwa, opis) VALUES (9, 'Strefa 9', 'Opis strefy 9');
INSERT INTO strefa (strefaid, nazwa, opis) VALUES (10, 'Strefa 10', 'Opis strefy 10');

INSERT INTO dok (dokid, nazwa, maks_nosnosc) VALUES (1, 'Dok 1', 4100);
INSERT INTO dok (dokid, nazwa, maks_nosnosc) VALUES (2, 'Dok 2', 4200);
INSERT INTO dok (dokid, nazwa, maks_nosnosc) VALUES (3, 'Dok 3', 4300);
INSERT INTO dok (dokid, nazwa, maks_nosnosc) VALUES (4, 'Dok 4', 4400);
INSERT INTO dok (dokid, nazwa, maks_nosnosc) VALUES (5, 'Dok 5', 4500);
INSERT INTO dok (dokid, nazwa, maks_nosnosc) VALUES (6, 'Dok 6', 4600);
INSERT INTO dok (dokid, nazwa, maks_nosnosc) VALUES (7, 'Dok 7', 4700);
INSERT INTO dok (dokid, nazwa, maks_nosnosc) VALUES (8, 'Dok 8', 4800);
INSERT INTO dok (dokid, nazwa, maks_nosnosc) VALUES (9, 'Dok 9', 4900);
INSERT INTO dok (dokid, nazwa, maks_nosnosc) VALUES (10, 'Dok 10', 5000);

INSERT INTO dok_nosnosc (dokid, maksymalnanosnosc, liczbamiejsccumowniczych) VALUES (1, 4100, 2);
INSERT INTO dok_nosnosc (dokid, maksymalnanosnosc, liczbamiejsccumowniczych) VALUES (2, 4200, 3);
INSERT INTO dok_nosnosc (dokid, maksymalnanosnosc, liczbamiejsccumowniczych) VALUES (3, 4300, 1);
INSERT INTO dok_nosnosc (dokid, maksymalnanosnosc, liczbamiejsccumowniczych) VALUES (4, 4400, 2);
INSERT INTO dok_nosnosc (dokid, maksymalnanosnosc, liczbamiejsccumowniczych) VALUES (5, 4500, 3);
INSERT INTO dok_nosnosc (dokid, maksymalnanosnosc, liczbamiejsccumowniczych) VALUES (6, 4600, 1);
INSERT INTO dok_nosnosc (dokid, maksymalnanosnosc, liczbamiejsccumowniczych) VALUES (7, 4700, 2);
INSERT INTO dok_nosnosc (dokid, maksymalnanosnosc, liczbamiejsccumowniczych) VALUES (8, 4800, 3);
INSERT INTO dok_nosnosc (dokid, maksymalnanosnosc, liczbamiejsccumowniczych) VALUES (9, 4900, 1);
INSERT INTO dok_nosnosc (dokid, maksymalnanosnosc, liczbamiejsccumowniczych) VALUES (10, 5000, 2);

INSERT INTO dok_stan (dokid, stan) VALUES (1, 'DOSTEPNY');
INSERT INTO dok_stan (dokid, stan) VALUES (2, 'ZAJETY');
INSERT INTO dok_stan (dokid, stan) VALUES (3, 'DOSTEPNY');
INSERT INTO dok_stan (dokid, stan) VALUES (4, 'ZAJETY');
INSERT INTO dok_stan (dokid, stan) VALUES (5, 'DOSTEPNY');
INSERT INTO dok_stan (dokid, stan) VALUES (6, 'ZAJETY');
INSERT INTO dok_stan (dokid, stan) VALUES (7, 'DOSTEPNY');
INSERT INTO dok_stan (dokid, stan) VALUES (8, 'ZAJETY');
INSERT INTO dok_stan (dokid, stan) VALUES (9, 'DOSTEPNY');
INSERT INTO dok_stan (dokid, stan) VALUES (10, 'ZAJETY');

INSERT INTO ciezarowka (ciezarowkaid, numerrejestracyjny, typ, nosnosc, status) VALUES (1, 'RZ0001A', 'Naczepa', 20.5, 'DOSTEPNA');
INSERT INTO ciezarowka (ciezarowkaid, numerrejestracyjny, typ, nosnosc, status) VALUES (2, 'RZ0002A', 'Naczepa', 21.0, 'W_TRASIE');
INSERT INTO ciezarowka (ciezarowkaid, numerrejestracyjny, typ, nosnosc, status) VALUES (3, 'RZ0003A', 'Naczepa', 21.5, 'DOSTEPNA');
INSERT INTO ciezarowka (ciezarowkaid, numerrejestracyjny, typ, nosnosc, status) VALUES (4, 'RZ0004A', 'Naczepa', 22.0, 'W_TRASIE');
INSERT INTO ciezarowka (ciezarowkaid, numerrejestracyjny, typ, nosnosc, status) VALUES (5, 'RZ0005A', 'Naczepa', 22.5, 'DOSTEPNA');
INSERT INTO ciezarowka (ciezarowkaid, numerrejestracyjny, typ, nosnosc, status) VALUES (6, 'RZ0006A', 'Naczepa', 23.0, 'W_TRASIE');
INSERT INTO ciezarowka (ciezarowkaid, numerrejestracyjny, typ, nosnosc, status) VALUES (7, 'RZ0007A', 'Naczepa', 23.5, 'DOSTEPNA');
INSERT INTO ciezarowka (ciezarowkaid, numerrejestracyjny, typ, nosnosc, status) VALUES (8, 'RZ0008A', 'Naczepa', 24.0, 'W_TRASIE');
INSERT INTO ciezarowka (ciezarowkaid, numerrejestracyjny, typ, nosnosc, status) VALUES (9, 'RZ0009A', 'Naczepa', 24.5, 'DOSTEPNA');
INSERT INTO ciezarowka (ciezarowkaid, numerrejestracyjny, typ, nosnosc, status) VALUES (10, 'RZ0010A', 'Naczepa', 25.0, 'W_TRASIE');

INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status) VALUES (1, 'STATEK_1', 'Kontenerowiec', 105
INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status) VALUES (2, 'STATEK_2', 'Kontenerowiec', 110
INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status) VALUES (3, 'STATEK_3', 'Kontenerowiec', 115
INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status) VALUES (4, 'STATEK_4', 'Kontenerowiec', 120
INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status) VALUES (5, 'STATEK_5', 'Kontenerowiec', 125
INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status) VALUES (6, 'STATEK_6', 'Kontenerowiec', 130
INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status) VALUES (7, 'STATEK_7', 'Kontenerowiec', 135
INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status) VALUES (8, 'STATEK_8', 'Kontenerowiec', 140
INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status) VALUES (9, 'STATEK_9', 'Kontenerowiec', 145
INSERT INTO statek (statekid, nazwa, typ, nosnosc, flaga, dataprzybycia, dataodplyniecia, status) VALUES (10, 'STATEK_10', 'Kontenerowiec', 1
```

```
INSERT INTO klient (klientid, nazwa, adres, numerkontaktu) VALUES (1, 'Klient 1', 'Miasto 1, ul. Portowa 1', '500000001');
INSERT INTO klient (klientid, nazwa, adres, numerkontaktu) VALUES (2, 'Klient 2', 'Miasto 2, ul. Portowa 2', '500000002');
INSERT INTO klient (klientid, nazwa, adres, numerkontaktu) VALUES (3, 'Klient 3', 'Miasto 3, ul. Portowa 3', '500000003');
INSERT INTO klient (klientid, nazwa, adres, numerkontaktu) VALUES (4, 'Klient 4', 'Miasto 4, ul. Portowa 4', '500000004');
INSERT INTO klient (klientid, nazwa, adres, numerkontaktu) VALUES (5, 'Klient 5', 'Miasto 5, ul. Portowa 5', '500000005');
INSERT INTO klient (klientid, nazwa, adres, numerkontaktu) VALUES (6, 'Klient 6', 'Miasto 6, ul. Portowa 6', '500000006');
INSERT INTO klient (klientid, nazwa, adres, numerkontaktu) VALUES (7, 'Klient 7', 'Miasto 7, ul. Portowa 7', '500000007');
INSERT INTO klient (klientid, nazwa, adres, numerkontaktu) VALUES (8, 'Klient 8', 'Miasto 8, ul. Portowa 8', '500000008');
INSERT INTO klient (klientid, nazwa, adres, numerkontaktu) VALUES (9, 'Klient 9', 'Miasto 9, ul. Portowa 9', '500000009');
INSERT INTO klient (klientid, nazwa, adres, numerkontaktu) VALUES (10, 'Klient 10', 'Miasto 10, ul. Portowa 10', '500000010');

INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga) VALUES (1, '20FT', 28100, 'Standard', 'Wymiary_1', 2210);
INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga) VALUES (2, '40FT', 28200, 'Standard', 'Wymiary_2', 2220);
INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga) VALUES (3, '20FT', 28300, 'Standard', 'Wymiary_3', 2230);
INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga) VALUES (4, '40FT', 28400, 'Standard', 'Wymiary_4', 2240);
INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga) VALUES (5, '20FT', 28500, 'Standard', 'Wymiary_5', 2250);
INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga) VALUES (6, '40FT', 28600, 'Standard', 'Wymiary_6', 2260);
INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga) VALUES (7, '20FT', 28700, 'Standard', 'Wymiary_7', 2270);
INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga) VALUES (8, '40FT', 28800, 'Standard', 'Wymiary_8', 2280);
INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga) VALUES (9, '20FT', 28900, 'Standard', 'Wymiary_9', 2290);
INSERT INTO kontener (kontenerid, typ, nosnosc, typkontenera, wymiary, waga) VALUES (10, '40FT', 29000, 'Standard', 'Wymiary_10', 2300);

INSERT INTO kontener_status (kontenerid, status) VALUES (1, 'WOLNY');
INSERT INTO kontener_status (kontenerid, status) VALUES (2, 'W_UZYCIU');
INSERT INTO kontener_status (kontenerid, status) VALUES (3, 'WOLNY');
INSERT INTO kontener_status (kontenerid, status) VALUES (4, 'W_UZYCIU');
INSERT INTO kontener_status (kontenerid, status) VALUES (5, 'WOLNY');
INSERT INTO kontener_status (kontenerid, status) VALUES (6, 'W_UZYCIU');
INSERT INTO kontener_status (kontenerid, status) VALUES (7, 'WOLNY');
INSERT INTO kontener_status (kontenerid, status) VALUES (8, 'W_UZYCIU');
INSERT INTO kontener_status (kontenerid, status) VALUES (9, 'WOLNY');
INSERT INTO kontener_status (kontenerid, status) VALUES (10, 'W_UZYCIU');
```

```
INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid) VALUES (1, 1);
INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid) VALUES (2, 2);
INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid) VALUES (3, 3);
INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid) VALUES (4, 4);
INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid) VALUES (5, 5);
INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid) VALUES (6, 6);
INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid) VALUES (7, 7);
INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid) VALUES (8, 8);
INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid) VALUES (9, 9);
INSERT INTO kontener_lokalizacja (kontenerid, lokalizacjaid) VALUES (10, 10);

INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid, harmonogram) VALUES (1, 'Imie1', 'Naz
INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid, harmonogram) VALUES (2, 'Imie2', 'Naz
INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid, harmonogram) VALUES (3, 'Imie3', 'Naz
INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid, harmonogram) VALUES (4, 'Imie4', 'Naz
INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid, harmonogram) VALUES (5, 'Imie5', 'Naz
INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid, harmonogram) VALUES (6, 'Imie6', 'Naz
INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid, harmonogram) VALUES (7, 'Imie7', 'Naz
INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid, harmonogram) VALUES (8, 'Imie8', 'Naz
INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid, harmonogram) VALUES (9, 'Imie9', 'Naz
INSERT INTO pracownik (pracownikid, imie, nazwisko, stanowisko, numerkontaktu, status, strefa_strefaid, harmonogram) VALUES (10, 'Imie10', 'N

INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany, datarozpoczecia, datazakonczenia, zadanie) VALUES (1, 1,
INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany, datarozpoczecia, datazakonczenia, zadanie) VALUES (2, 2,
INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany, datarozpoczecia, datazakonczenia, zadanie) VALUES (3, 3,
INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany, datarozpoczecia, datazakonczenia, zadanie) VALUES (4, 4,
INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany, datarozpoczecia, datazakonczenia, zadanie) VALUES (5, 5,
INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany, datarozpoczecia, datazakonczenia, zadanie) VALUES (6, 6,
INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany, datarozpoczecia, datazakonczenia, zadanie) VALUES (7, 7,
INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany, datarozpoczecia, datazakonczenia, zadanie) VALUES (8, 8,
INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany, datarozpoczecia, datazakonczenia, zadanie) VALUES (9, 9,
INSERT INTO harmonogrampracownika (harmonogrampracownikaid, pracownikid, typzmiany, datarozpoczecia, datazakonczenia, zadanie) VALUES (10, 10
```

```
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (1, 1, 1);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (2, 2, 2);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (3, 3, 3);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (4, 4, 4);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (5, 5, 5);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (6, 6, 6);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (7, 7, 7);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (8, 8, 8);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (9, 9, 9);
INSERT INTO pracownik_strefa (id, pracownikid, strefaid) VALUES (10, 10, 10);

INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status) VALUES (1, 1, 42, 'STS', 'SPRAWNY');
INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status) VALUES (2, 2, 44, 'RTG', 'SPRAWNY');
INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status) VALUES (3, 3, 46, 'STS', 'SERWIS');
INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status) VALUES (4, 4, 48, 'RTG', 'SPRAWNY');
INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status) VALUES (5, 5, 50, 'STS', 'SPRAWNY');
INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status) VALUES (6, 6, 52, 'RTG', 'SERWIS');
INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status) VALUES (7, 7, 54, 'STS', 'SPRAWNY');
INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status) VALUES (8, 8, 56, 'RTG', 'SPRAWNY');
INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status) VALUES (9, 9, 58, 'STS', 'SERWIS');
INSERT INTO dzwig (dzwigid, strefaid, maksymalnanosnosc, typ, status) VALUES (10, 10, 60, 'RTG', 'SPRAWNY');

INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc, zajetaprzestrzen, status) VALUES (1, 1, 'Standard', 11000, 1050,
INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc, zajetaprzestrzen, status) VALUES (2, 2, 'Standard', 12000, 1100,
INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc, zajetaprzestrzen, status) VALUES (3, 3, 'Standard', 13000, 1150,
INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc, zajetaprzestrzen, status) VALUES (4, 4, 'Standard', 14000, 1200,
INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc, zajetaprzestrzen, status) VALUES (5, 5, 'Standard', 15000, 1250,
INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc, zajetaprzestrzen, status) VALUES (6, 6, 'Standard', 16000, 1300,
INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc, zajetaprzestrzen, status) VALUES (7, 7, 'Standard', 17000, 1350,
INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc, zajetaprzestrzen, status) VALUES (8, 8, 'Standard', 18000, 1400,
INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc, zajetaprzestrzen, status) VALUES (9, 9, 'Standard', 19000, 1450,
INSERT INTO magazyn (magazynid, strefaid, typmagazynu, maksymalnapojemnosc, zajetaprzestrzen, status) VALUES (10, 10, 'Standard', 20000, 1500
```

```
INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca) VALUES (1, TO_DATE('2025-01-01','YYYY-MM-DD'), TO_DATE('2026-01-01',
INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca) VALUES (2, TO_DATE('2025-01-01','YYYY-MM-DD'), TO_DATE('2026-01-01',
INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca) VALUES (3, TO_DATE('2025-01-01','YYYY-MM-DD'), TO_DATE('2026-01-01',
INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca) VALUES (4, TO_DATE('2025-01-01','YYYY-MM-DD'), TO_DATE('2026-01-01',
INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca) VALUES (5, TO_DATE('2025-01-01','YYYY-MM-DD'), TO_DATE('2026-01-01',
INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca) VALUES (6, TO_DATE('2025-01-01','YYYY-MM-DD'), TO_DATE('2026-01-01',
INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca) VALUES (7, TO_DATE('2025-01-01','YYYY-MM-DD'), TO_DATE('2026-01-01',
INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca) VALUES (8, TO_DATE('2025-01-01','YYYY-MM-DD'), TO_DATE('2026-01-01',
INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca) VALUES (9, TO_DATE('2025-01-01','YYYY-MM-DD'), TO_DATE('2026-01-01',
INSERT INTO ubezpieczenie (ubezpieczenieid, datapoczatku, datakonca) VALUES (10, TO_DATE('2025-01-01','YYYY-MM-DD'), TO_DATE('2026-01-01'

INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid) VALUES ('TYP1', 60000, 1);
INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid) VALUES ('TYP2', 70000, 2);
INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid) VALUES ('TYP3', 80000, 3);
INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid) VALUES ('TYP4', 90000, 4);
INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid) VALUES ('TYP5', 100000, 5);
INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid) VALUES ('TYP6', 110000, 6);
INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid) VALUES ('TYP7', 120000, 7);
INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid) VALUES ('TYP8', 130000, 8);
INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid) VALUES ('TYP9', 140000, 9);
INSERT INTO typ_ubezpieczenia (typubezpieczenia, kwota, ubezpieczenie_ubezpieczenieid) VALUES ('TYP10', 150000, 10);

INSERT INTO warunki_ubezpieczenia (typubezpieczenia, warunki) VALUES ('TYP1', 'WARUNEK1');
INSERT INTO warunki_ubezpieczenia (typubezpieczenia, warunki) VALUES ('TYP2', 'WARUNEK2');
INSERT INTO warunki_ubezpieczenia (typubezpieczenia, warunki) VALUES ('TYP3', 'WARUNEK3');
INSERT INTO warunki_ubezpieczenia (typubezpieczenia, warunki) VALUES ('TYP4', 'WARUNEK4');
INSERT INTO warunki_ubezpieczenia (typubezpieczenia, warunki) VALUES ('TYP5', 'WARUNEK5');
INSERT INTO warunki_ubezpieczenia (typubezpieczenia, warunki) VALUES ('TYP6', 'WARUNEK6');
INSERT INTO warunki_ubezpieczenia (typubezpieczenia, warunki) VALUES ('TYP7', 'WARUNEK7');
INSERT INTO warunki_ubezpieczenia (typubezpieczenia, warunki) VALUES ('TYP8', 'WARUNEK8');
INSERT INTO warunki_ubezpieczenia (typubezpieczenia, warunki) VALUES ('TYP9', 'WARUNEK9');
INSERT INTO warunki_ubezpieczenia (typubezpieczenia, warunki) VALUES ('TYP10', 'WARUNEK10');
```

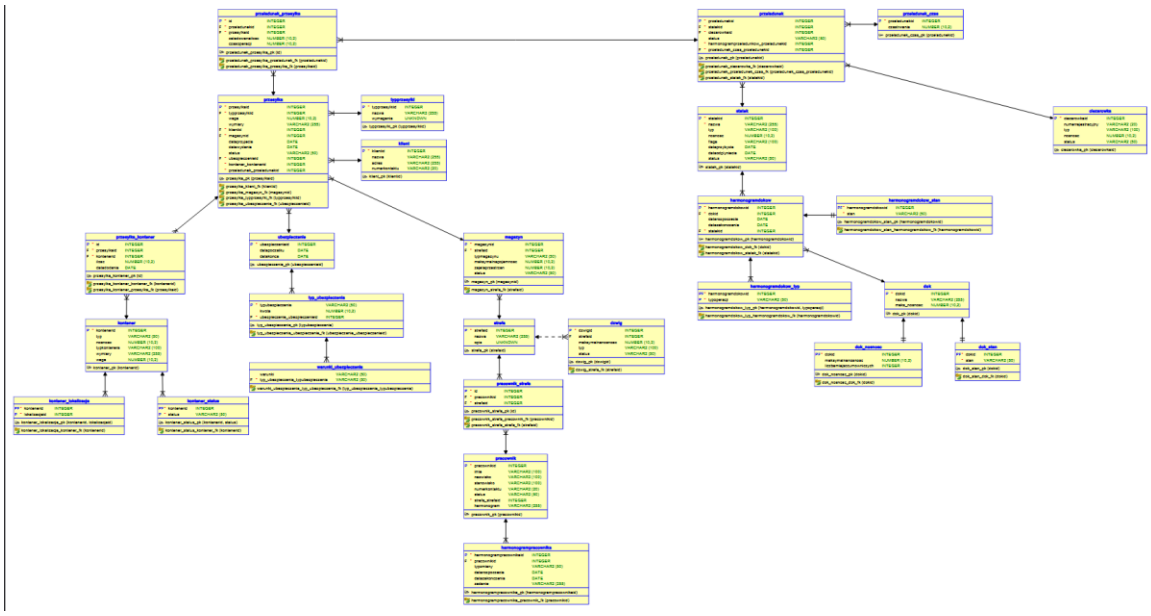
```
INSERT INTO typprzesylki (typprzesylkiid, nazwa, wymagania) VALUES (1, 'TypPrzesylki 1', 'Wymagania 1');
INSERT INTO typprzesylki (typprzesylkiid, nazwa, wymagania) VALUES (2, 'TypPrzesylki 2', 'Wymagania 2');
INSERT INTO typprzesylki (typprzesylkiid, nazwa, wymagania) VALUES (3, 'TypPrzesylki 3', 'Wymagania 3');
INSERT INTO typprzesylki (typprzesylkiid, nazwa, wymagania) VALUES (4, 'TypPrzesylki 4', 'Wymagania 4');
INSERT INTO typprzesylki (typprzesylkiid, nazwa, wymagania) VALUES (5, 'TypPrzesylki 5', 'Wymagania 5');
INSERT INTO typprzesylki (typprzesylkiid, nazwa, wymagania) VALUES (6, 'TypPrzesylki 6', 'Wymagania 6');
INSERT INTO typprzesylki (typprzesylkiid, nazwa, wymagania) VALUES (7, 'TypPrzesylki 7', 'Wymagania 7');
INSERT INTO typprzesylki (typprzesylkiid, nazwa, wymagania) VALUES (8, 'TypPrzesylki 8', 'Wymagania 8');
INSERT INTO typprzesylki (typprzesylkiid, nazwa, wymagania) VALUES (9, 'TypPrzesylki 9', 'Wymagania 9');
INSERT INTO typprzesylki (typprzesylkiid, nazwa, wymagania) VALUES (10, 'TypPrzesylki 10', 'Wymagania 10');

INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia, datazakonczenia, statekid) VALUES (1, 1, TO_DATE('2025-01-03','YYYY
INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia, datazakonczenia, statekid) VALUES (2, 2, TO_DATE('2025-01-05','YYYY
INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia, datazakonczenia, statekid) VALUES (3, 3, TO_DATE('2025-01-07','YYYY
INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia, datazakonczenia, statekid) VALUES (4, 4, TO_DATE('2025-01-09','YYYY
INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia, datazakonczenia, statekid) VALUES (5, 5, TO_DATE('2025-01-11','YYYY
INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia, datazakonczenia, statekid) VALUES (6, 6, TO_DATE('2025-01-13','YYYY
INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia, datazakonczenia, statekid) VALUES (7, 7, TO_DATE('2025-01-15','YYYY
INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia, datazakonczenia, statekid) VALUES (8, 8, TO_DATE('2025-01-17','YYYY
INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia, datazakonczenia, statekid) VALUES (9, 9, TO_DATE('2025-01-19','YYYY
INSERT INTO harmonogramdokow (harmonogramdokowid, dokid, datarozpoczecia, datazakonczenia, statekid) VALUES (10, 10, TO_DATE('2025-01-21','YY

INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan) VALUES (1, 'ZAPLANOWANY');
INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan) VALUES (2, 'ZAKONCZONY');
INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan) VALUES (3, 'ZAPLANOWANY');
INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan) VALUES (4, 'ZAKONCZONY');
INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan) VALUES (5, 'ZAPLANOWANY');
INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan) VALUES (6, 'ZAKONCZONY');
INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan) VALUES (7, 'ZAPLANOWANY');
INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan) VALUES (8, 'ZAKONCZONY');
INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan) VALUES (9, 'ZAPLANOWANY');
INSERT INTO harmonogramdokow_stan (harmonogramdokowid, stan) VALUES (10, 'ZAKONCZONY');
```

```
INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji) VALUES (1, 'ZALADUNEK');
INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji) VALUES (2, 'SERWIS');
INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji) VALUES (3, 'ROZLADUNEK');
INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji) VALUES (4, 'ZALADUNEK');
INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji) VALUES (5, 'SERWIS');
INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji) VALUES (6, 'ROZLADUNEK');
INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji) VALUES (7, 'ZALADUNEK');
INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji) VALUES (8, 'SERWIS');
INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji) VALUES (9, 'ROZLADUNEK');
INSERT INTO harmonogramdokow_typ (harmonogramdokowid, typoperacji) VALUES (10, 'ZALADUNEK');

INSERT INTO przeladunek_czas (przeladunekid, czastrwania) VALUES (1, 4.5);
INSERT INTO przeladunek_czas (przeladunekid, czastrwania) VALUES (2, 5.0);
INSERT INTO przeladunek_czas (przeladunekid, czastrwania) VALUES (3, 5.5);
INSERT INTO przeladunek_czas (przeladunekid, czastrwania) VALUES (4, 6.0);
INSERT INTO przeladunek_czas (przeladunekid, czastrwania) VALUES (5, 6.5);
INSERT INTO przeladunek_czas (przeladunekid, czastrwania) VALUES (6, 7.0);
INSERT INTO przeladunek_czas (przeladunekid, czastrwania) VALUES (7, 7.5);
INSERT INTO przeladunek_czas (przeladunekid, czastrwania) VALUES (8, 8.0);
INSERT INTO przeladunek_czas (przeladunekid, czastrwania) VALUES (9, 8.5);
INSERT INTO przeladunek_czas (przeladunekid, czastrwania) VALUES (10, 9.0);

INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status, harmonogramdokowid, przeladunek_czas_przeladunekid) VALUES (1, 1, 1,
INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status, harmonogramdokowid, przeladunek_czas_przeladunekid) VALUES (2, 2, 2,
INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status, harmonogramdokowid, przeladunek_czas_przeladunekid) VALUES (3, 3, 3,
INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status, harmonogramdokowid, przeladunek_czas_przeladunekid) VALUES (4, 4, 4,
INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status, harmonogramdokowid, przeladunek_czas_przeladunekid) VALUES (5, 5, 5,
INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status, harmonogramdokowid, przeladunek_czas_przeladunekid) VALUES (6, 6, 6,
INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status, harmonogramdokowid, przeladunek_czas_przeladunekid) VALUES (7, 7, 7,
INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status, harmonogramdokowid, przeladunek_czas_przeladunekid) VALUES (8, 8, 8,
INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status, harmonogramdokowid, przeladunek_czas_przeladunekid) VALUES (9, 9, 9,
INSERT INTO przeladunek (przeladunekid, statekid, ciezarowkaid, status, harmonogramdokowid, przeladunek_czas_przeladunekid) VALUES (10, 10, 1
```

```
INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania) VALUES (1, 1, 1, 2, TO_DATE('2025-01-03','YYYY-MM-DD'));
INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania) VALUES (2, 2, 2, 3, TO_DATE('2025-01-05','YYYY-MM-DD'));
INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania) VALUES (3, 3, 3, 4, TO_DATE('2025-01-07','YYYY-MM-DD'));
INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania) VALUES (4, 4, 4, 5, TO_DATE('2025-01-09','YYYY-MM-DD'));
INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania) VALUES (5, 5, 5, 1, TO_DATE('2025-01-11','YYYY-MM-DD'));
INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania) VALUES (6, 6, 6, 2, TO_DATE('2025-01-13','YYYY-MM-DD'));
INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania) VALUES (7, 7, 7, 3, TO_DATE('2025-01-15','YYYY-MM-DD'));
INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania) VALUES (8, 8, 8, 4, TO_DATE('2025-01-17','YYYY-MM-DD'));
INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania) VALUES (9, 9, 9, 5, TO_DATE('2025-01-19','YYYY-MM-DD'));
INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, ilosc, datadodania) VALUES (10, 10, 10, 1, TO_DATE('2025-01-21','YYYY-MM-DD'));

INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc, czasoperacji) VALUES (1, 1, 1, 2, 1.7);
INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc, czasoperacji) VALUES (2, 2, 2, 3, 1.9);
INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc, czasoperacji) VALUES (3, 3, 3, 1, 2.1);
INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc, czasoperacji) VALUES (4, 4, 4, 2, 2.3);
INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc, czasoperacji) VALUES (5, 5, 5, 3, 2.5);
INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc, czasoperacji) VALUES (6, 6, 6, 1, 2.7);
INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc, czasoperacji) VALUES (7, 7, 7, 2, 2.9);
INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc, czasoperacji) VALUES (8, 8, 8, 3, 3.1);
INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc, czasoperacji) VALUES (9, 9, 9, 1, 3.3);
INSERT INTO przeladunek_przesylka (id, przeladunekid, przesylkaid, zaladowanailosc, czasoperacji) VALUES (10, 10, 10, 2, 3.5);
```

2.5 Wygląd bazy

Wszystkie tabele bazy danych zostały utworzone za pomocą języka SQL. Za pomocą poniższego schematu:



3. Implementacja pakietów

3.1. Pakiet operacje doku

Pakiet `operacje_doku_prac` został zaprojektowany w celu centralizacji logiki biznesowej związanej z obsługą doków portowych, statków, pracowników oraz zasobów technicznych. Jego głównym zadaniem jest zapewnienie spójnych zasad walidacji oraz wykonywania operacji operacyjnych w bazie danych, tak aby wszystkie procesy związane z harmonogramowaniem i obsługą doku były realizowane w jednym, kontrolowanym miejscu.

W pierwszej części pakietu znajdują się funkcje sprawdzające, których celem jest weryfikacja warunków niezbędnych do wykonania operacji. Funkcja `dok_istnieje` sprawdza, czy dok o podanym identyfikatorze istnieje w systemie, co zapobiega wykonywaniu operacji na nieistniejących danych. Funkcja `dok_aktywny` weryfikuje, czy dany dok jest aktualnie aktywny i może być wykorzystywany do obsługi statków. Funkcja `dok_wolny_w_dniu` sprawdza dostępność doku w określonym dniu, analizując harmonogram zajętości i eliminując konflikty terminów. Funkcja `czy_mozna_obslugiwac_dok` pełni rolę funkcji nadrzędnej – wykorzystuje wcześniejsze funkcje sprawdzające i na ich podstawie określa, czy w danym dniu spełnione są wszystkie warunki do rozpoczęcia obsługi doku.

Druga część pakietu zawiera procedury operacyjne, które odpowiadają za faktyczne modyfikowanie danych w bazie. Procedura `przypisz_statek_do_doku` realizuje proces przypisania statku do konkretnego doku w określonym przedziale czasowym, tworząc odpowiedni wpis w harmonogramie i zakładając, że warunki dostępności zostały wcześniej sprawdzone. Procedura `zakoncz_obsluge_doku` służy do zakończenia obsługi doku poprzez aktualizację daty zakończenia w harmonogramie, co formalnie zwalnia dok do dalszego wykorzystania. Procedura `ustaw_status_dzwigu` umożliwia zmianę statusu dźwigu, na przykład na „AKTYWNY” lub „NIEAKTYWNY”, co odzwierciedla bieżącą dostępność zasobu technicznego. Procedura `dodaj_pracownika_do_grafiku` odpowiada za planowanie pracy personelu – dodaje pracownika do grafiku na określony typ zmiany, przedział czasowy oraz zadanie, zapewniając powiązanie zasobów ludzkich z operacjami wykonywanymi w doku.

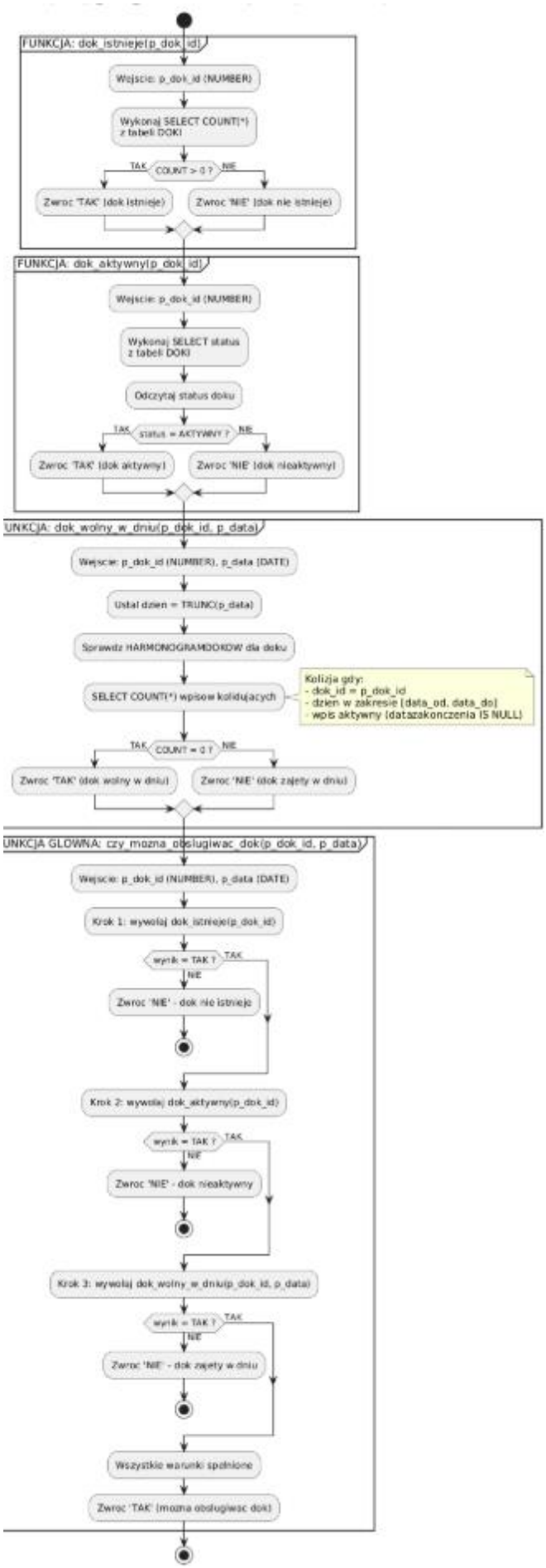
Cały pakiet stanowi logiczną warstwę pośrednią pomiędzy aplikacją (np. Oracle APEX) a tabelami bazy danych. Dzięki temu reguły biznesowe są skupione w jednym miejscu, kod jest czytelniejszy i łatwiejszy w utrzymaniu, a ryzyko niespójnych danych oraz błędów logicznych zostaje znacząco ograniczone.

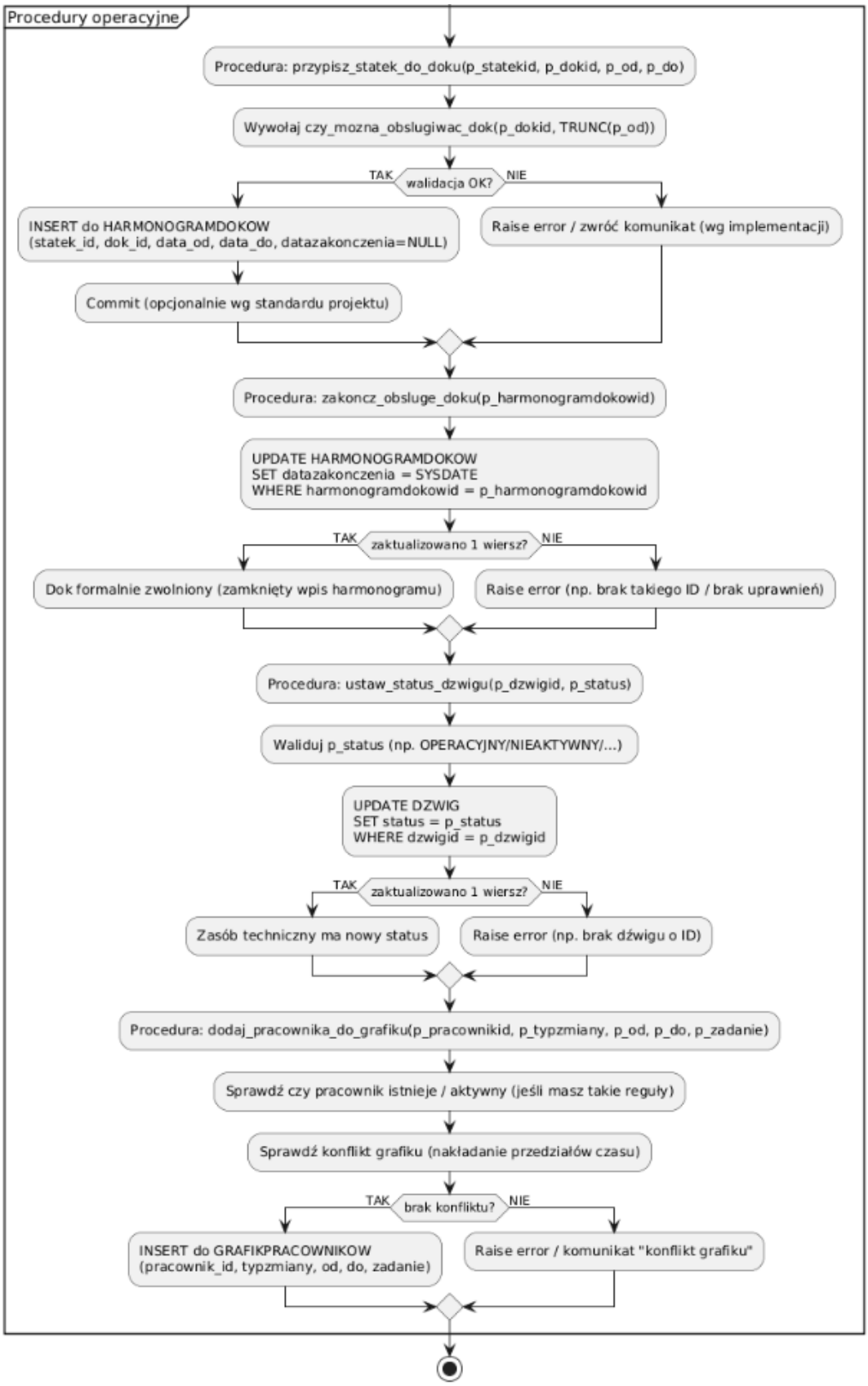
OPERACJE_DOKU_PRAC

Specification | Body | Dependencies | Errors | Grants

Download | Save and Compile | Drop | Refresh

```
1  create or replace PACKAGE operacje_doku_prac AS
2    -- FUNKCJE SPRAWDZAJACE
3    FUNCTION dok_istnieje(
4      p_dok_id IN NUMBER
5    ) RETURN VARCHAR2;
6    FUNCTION dok_aktywny(
7      p_dok_id IN NUMBER
8    ) RETURN VARCHAR2;
9    FUNCTION dok_wolny_w_dniu(
10     p_dok_id IN NUMBER,
11     p_data   IN DATE
12   ) RETURN VARCHAR2;
13   FUNCTION czy_mozna_obslugiwac_dok(
14     p_dok_id IN NUMBER,
15     p_data   IN DATE
16   ) RETURN VARCHAR2;
17   -- PROCEDURY OPERACYJNE
18   PROCEDURE przypisz_statek_do_doku(
19     p_statekid IN NUMBER,
20     p_dokid    IN NUMBER,
21     p_od       IN DATE,
22     p_do       IN DATE
23   );
24   PROCEDURE zakoncz_obsluge_doku(
25     p_harmonogramdokowid IN NUMBER
26   );
27   PROCEDURE ustaw_status_dzwigu(
28     p_dzwigid IN NUMBER,
29     p_status  IN VARCHAR2
30   );
31   PROCEDURE dodaj_pracownika_do_grafiku(
32     p_pracownikid IN NUMBER,
33     p_typzmiany   IN VARCHAR2,
34     p_od          IN DATE,
35     p_do          IN DATE,
36     p_zadanie     IN VARCHAR2
37   );
38 END operacje_doku_prac;
39 /
```


FUNKCJA: dok_istnieje(p_dok_id)
Wejscie: p_dok_id (NUMBER)
Wykonaj SELECT COUNT(*) z tabeli DOKI
COUNT > 0 ?
TAK
NIE
Zwroc 'TAK' (dok istnieje)
Zwroc 'NIE' (dok nie istnieje)
FUNKCJA: dok_aktywny(p_dok_id)
Wejscie: p_dok_id (NUMBER)
Wykonaj SELECT status z tabeli DOKI
Odczytaj status doku
status = AKTYWNY ?
TAK
NIE
Zwroc 'TAK' (dok aktywny)
Zwroc 'NIE' (dok nieaktywny)
FUNKCJA: dok_wolny_w_dniu(p_dok_id, p_data)
Wejscie: p_dok_id (NUMBER), p_data (DATE)
Ustal dzien = TRUNC(p_data)
Sprawdz HARMONOGRAMDOKOW dla doku
SELECT COUNT(*) wpisow kolidujacych
Kolizja gdy:
- dok_id = p_dok_id
- dzien w zakresie [data_od, data_do]
- wpis aktywny (datazakonczenia IS NULL)
COUNT = 0 ?
TAK
NIE
Zwroc 'TAK' (dok wolny w dniu)
Zwroc 'NIE' (dok zajety w dniu)
FUNKCJA GLOWNA: czy_mozna_obslugiwac_dok(p_dok_id, p_data)
Wejscie: p_dok_id (NUMBER), p_data (DATE)
Krok 1: wywolaj dok_istnieje(p_dok_id)
wynik = TAK ?
TAK
NIE
Zwroc 'NIE' - dok nie istnieje
Krok 2: wywolaj dok_aktywny(p_dok_id)
wynik = TAK ?
TAK
NIE
Zwroc 'NIE' - dok nieaktywny
Krok 3: wywolaj dok_wolny_w_dniu(p_dok_id, p_data)
wynik = TAK ?
TAK
NIE
Zwroc 'NIE' - dok zajety w dniu
Wszystkie warunki spełnione
Zwroc 'TAK' (mozna obslugiwac dok)


Procedury operacyjne
Procedura: przypisz_statek_do_doku(p_statekid, p_dokid, p_od, p_do)
Wywołaj czy_mozna_obslugiwac_dok(p_dokid, TRUNC(p_od))
walidacja OK?
TAK
NIE
INSERT do HARMONOGRAMDOKOW
(statek_id, dok_id, data_od, data_do, datazakonczenia=NULL)
Raise error / zwróć komunikat (wg implementacji)
Commit (opcjonalnie wg standardu projektu)
Procedura: zakoncz_obsluge_doku(p_harmonogramdokowid)
UPDATE HARMONOGRAMDOKOW
SET datazakonczenia = SYSDATE
WHERE harmonogramdokowid = p_harmonogramdokowid
zaktualizowano 1 wiersz?
TAK
NIE
Dok formalnie zwolniony (zamknięty wpis harmonogramu)
Raise error (np. brak takiego ID / brak uprawnień)
Procedura: ustaw_status_dzwigu(p_dzwigid, p_status)
Waliduj p_status (np. OPERACYJNY/NIEAKTYWNY/...)
UPDATE DZWIG
SET status = p_status
WHERE dzwigid = p_dzwigid
zaktualizowano 1 wiersz?
TAK
NIE
Zasób techniczny ma nowy status
Raise error (np. brak dźwigu o ID)
Procedura: dodaj_pracownika_do_grafiku(p_pracownikid, p_typzmiany, p_od, p_do, p_zadanie)
Sprawdź czy pracownik istnieje / aktywny (jeśli masz takie reguły)
Sprawdź konflikt grafiku (nakładanie przedziałów czasu)
brak konfliktu?
TAK
NIE
INSERT do GRAFIKPRACOWNIKOW
(pracownik_id, typzmiany, od, do, zadanie)
Raise error / komunikat "konflikt grafiku"

Funkcja DOK_AKTYWNY

```
create or replace FUNCTION dok_aktywny(
  p_dok_id IN NUMBER
) RETURN VARCHAR2
IS
  v_nosnosc NUMBER;
BEGIN
  SELECT maks_nosnosc
    INTO v_nosnosc
    FROM dok
   WHERE dokid = p_dok_id;

  RETURN CASE
           WHEN NVL(v_nosnosc, 0) > 0 THEN 'TAK'
           ELSE 'NIE'
         END;

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN 'NIE';
END;
/
```

Funkcja dok_aktywny służy do sprawdzenia, czy wskazany dok może być uznany za aktywny z punktu widzenia technicznego. Jako kryterium aktywności przyjęto istnienie dodatniej maksymalnej nośności doku. Funkcja przyjmuje jako parametr identyfikator doku, następnie pobiera z tabeli DOK wartość kolumny MAKS_NOSNOSC dla wskazanego rekordu i sprawdza, czy jej wartość jest większa od zera. Jeżeli warunek ten jest spełniony, funkcja zwraca wartość 'TAK', w przeciwnym przypadku zwraca 'NIE'. W sytuacji, gdy w tabeli nie istnieje dok o podanym identyfikatorze, obsługiwany jest wyjątek NO_DATA_FOUND, a funkcja również zwraca wartość 'NIE'. Funkcja wykorzystywana jest jako element logiki biznesowej w procesie weryfikacji możliwości obsługi doku, w szczególności w funkcji czy_mozna_obslugiwac_dok oraz w walidacjach aplikacji Oracle APEX.

Funkcja DOK_ISTNIEJE

```
create or replace FUNCTION dok_istnieje(
  p_dok_id IN NUMBER
) RETURN VARCHAR2
IS
  v_cnt NUMBER;
BEGIN
  SELECT COUNT(*)
    INTO v_cnt
    FROM dok
   WHERE dokid = p_dok_id;

  RETURN CASE WHEN v_cnt = 1 THEN 'TAK' ELSE 'NIE' END;
END;
/
```

\Funkcja dok_istnieje służy do sprawdzenia, czy w bazie danych istnieje dok o podanym identyfikatorze. Funkcja przyjmuje jako parametr identyfikator doku, a następnie wykonuje zapytanie zliczające liczbę rekordów w tabeli DOK o wskazanym identyfikatorze. Jeżeli liczba znalezionych rekordów jest równa jeden, funkcja zwraca wartość 'TAK', co oznacza, że dok istnieje. W przeciwnym przypadku, gdy rekord nie zostanie znaleziony lub wystąpi inna liczba rekordów, funkcja zwraca wartość 'NIE'. Funkcja pełni rolę pomocniczą w logice biznesowej aplikacji i jest wykorzystywana do weryfikacji poprawności danych wejściowych, w szczególności w procesach związanych z obsługą doku oraz w funkcji czy_mozna_obslugiwac_dok.

FUNKCJA DOK_WOLNY_W_DNIU

```oracle_plsql
create or replace FUNCTION dok_wolny_w_dniu(
  p_dok_id IN NUMBER,
  p_data   IN DATE
) RETURN VARCHAR2
IS
  v_cnt NUMBER;
BEGIN
  SELECT COUNT(*)
    INTO v_cnt
    FROM harmonogramdokow hd
   WHERE hd.dokid = p_dok_id
     AND p_data >= hd.datarozpoczecia
     AND p_data <  NVL(hd.datazakonczenia, p_data + 1);

  RETURN CASE WHEN v_cnt = 0 THEN 'TAK' ELSE 'NIE' END;
END;
/
```

Funkcja dok_wolny_w_dniu służy do sprawdzenia, czy wskazany dok jest dostępny do obsługi w podanym dniu. Funkcja przyjmuje jako parametry identyfikator doku oraz datę, dla której wykonywane jest sprawdzenie. Następnie analizuje tabelę HARMONOGRAMDOKOW w celu ustalenia, czy w danym dniu istnieje już wpis dotyczący obsługi tego doku. Sprawdzenie polega na zliczeniu rekordów, dla których podana data mieści się w przedziale pomiędzy datą rozpoczęcia a datą zakończenia obsługi doku. Jeżeli liczba takich rekordów wynosi zero, funkcja zwraca wartość 'TAK', co oznacza, że dok jest wolny w danym dniu. W przeciwnym przypadku, gdy istnieje co najmniej jeden harmonogram obejmujący wskazaną datę, funkcja zwraca wartość 'NIE'. Funkcja stanowi element logiki biznesowej systemu i jest wykorzystywana do zapobiegania jednoczesnej obsłudze jednego doku w tym samym czasie, w szczególności w funkcji czy_mozna_obslugiwac_dok oraz w walidacjach aplikacji Oracle APEX

FUNCKJA CZY_MOZNA_OBSLUGIWAC_DOK

```
create or replace FUNCTION czy_mozna_obslugiwac_dok(
  p_dok_id IN NUMBER,
  p_data   IN DATE
) RETURN VARCHAR2
IS
  v_maks_nosnosc   NUMBER;
  v_akt_nosnosc    NUMBER;
BEGIN
  -- WARUNEK 0: data nie moze byc z przeszlosci
  IF p_data < TRUNC(SYSDATE) THEN
    RETURN 'NIE';
  END IF;

  -- WARUNEK 1: dok istnieje
  IF dok_istnieje(p_dok_id) <> 'TAK' THEN
    RETURN 'NIE';
  END IF;

  -- WARUNEK 2: dok aktywny (wg MAKS_NOSNOSC)
  IF dok_aktywny(p_dok_id) <> 'TAK' THEN
    RETURN 'NIE';
  END IF;

  -- WARUNEK 3: dok wolny w danym dniu
  IF dok_wolny_w_dniu(p_dok_id, p_data) <> 'TAK' THEN
    RETURN 'NIE';
  END IF;

  -- WARUNEK 4: sprawdzenie nosnosci doku
  SELECT maks_nosnosc
    INTO v_maks_nosnosc
    FROM dok
   WHERE dokid = p_dok_id;

  SELECT NVL(SUM(s.nosnosc), 0)
    INTO v_akt_nosnosc
    FROM harmonogramdokow hd
    JOIN statek s ON s.statekid = hd.statekid
   WHERE hd.dokid = p_dok_id
     AND p_data >= hd.datarozpoczecia
     AND p_data <  NVL(hd.datazakonczenia, p_data + 1);

  IF v_akt_nosnosc > v_maks_nosnosc THEN
    RETURN 'NIE';
  END IF;

  RETURN 'TAK';

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN 'NIE';
END;
/
```

Funkcja czy_mozna_obslugiwac_dok stanowi główny element logiki decyzyjnej dotyczącej możliwości obsługi doku w określonym dniu. Jej zadaniem jest kompleksowa weryfikacja spełnienia warunków biznesowych niezbędnych do rozpoczęcia obsługi doku. Funkcja przyjmuje jako parametry identyfikator doku oraz datę planowanej obsługi, po czym kolejno sprawdza poprawność tych danych. W pierwszym kroku weryfikuje, czy podana data nie jest datą z przeszłości, co eliminuje możliwość planowania obsługi wstecz. Następnie sprawdzane jest istnienie doku w bazie danych przy użyciu funkcji dok_istnieje, jego aktywność techniczna poprzez funkcję dok_aktywny oraz dostępność czasowa w danym dniu za pomocą funkcji dok_wolny_w_dniu. Dodatkowo funkcja kontroluje, czy łączna nośność statków przypisanych do doku w podanym dniu nie przekracza maksymalnej dopuszczalnej nośności doku. Jeżeli którykolwiek z warunków nie zostanie spełniony, funkcja zwraca wartość 'NIE'. W przypadku spełnienia wszystkich kryteriów funkcja zwraca wartość 'TAK', co oznacza możliwość obsługi doku w zadanym terminie. Funkcja jest wykorzystywana jako centralny mechanizm walidacyjny w aplikacji Oracle APEX oraz w procedurach operacyjnych związanych z obsługą doku.

- Procedury

PROCEDURA DODAJ_PRACOWNIKA_DO_GRAFIKU

```oracle_plsql
create or replace PROCEDURE dodaj_pracownika_do_grafiku (
  p_pracownikid_txt IN VARCHAR2,
  p_typzmiany       IN VARCHAR2,
  p_od              IN DATE,
  p_do              IN DATE,
  p_zadanie         IN VARCHAR2
) IS
  -- KOLEKCJA:
  TYPE t_id_tab IS TABLE OF harmonogrampracownika.harmonogrampracownikaid%TYPE;
  v_koliduje t_id_tab;

  -- REF CURSOR
  TYPE t_refcur IS REF CURSOR;
  rc_kolizje t_refcur;

  v_pracownikid NUMBER;
  v_cnt NUMBER;
  v_one_id harmonogrampracownika.harmonogrampracownikaid%TYPE;

BEGIN
  -- 1) Walidacje
  IF p_pracownikid_txt IS NULL OR TRIM(p_pracownikid_txt) IS NULL THEN
    RAISE_APPLICATION_ERROR(-20040, 'PRACOWNIKID jest wymagany.');
  END IF;

  IF NOT REGEXP_LIKE(TRIM(p_pracownikid_txt), '^\d+$') THEN
    RAISE_APPLICATION_ERROR(-20041, 'PRACOWNIKID musi byc liczba calkowita.');
  END IF;

  v_pracownikid := TO_NUMBER(TRIM(p_pracownikid_txt));

  IF p_typzmiany IS NULL OR TRIM(p_typzmiany) IS NULL THEN
    RAISE_APPLICATION_ERROR(-20042, 'TYPZMIANY jest wymagany.');
  END IF;

  IF p_od IS NULL OR p_do IS NULL THEN
    RAISE_APPLICATION_ERROR(-20043, 'DATA_OD i DATA_DO sa wymagane.');
  END IF;
```

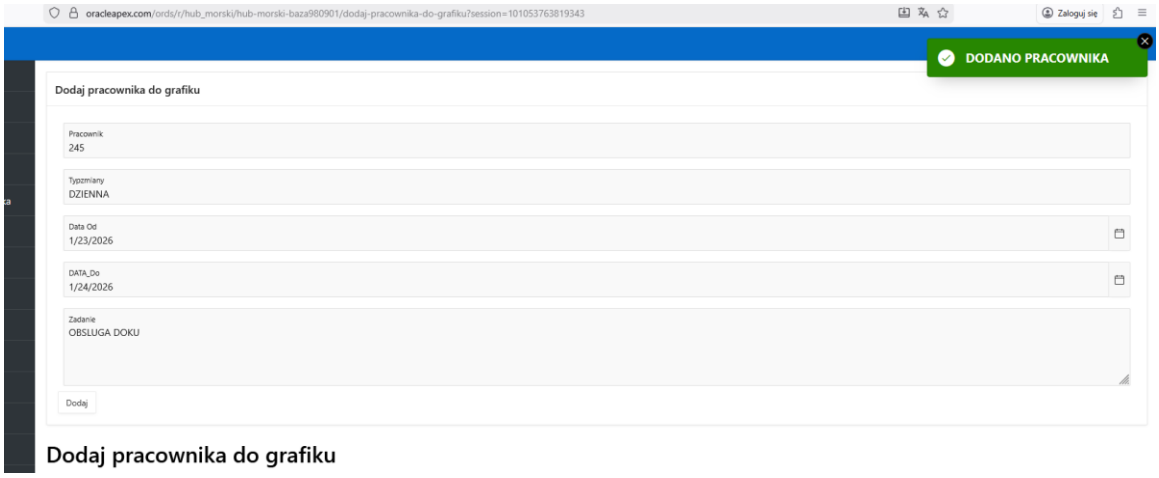
```oracle_plsql
IF p_do <= p_od THEN
  RAISE_APPLICATION_ERROR(-20044, 'DATA_DO musi byc pozniejsza niz DATA_OD.');
END IF;


SELECT COUNT(*)
  INTO v_cnt
  FROM pracownik
 WHERE pracownikid = v_pracownikid;

IF v_cnt = 0 THEN
  RAISE_APPLICATION_ERROR(-20045, 'Nie istnieje taki PRACOWNIKID.');
END IF;
```

```
  v_koliduje := t_id_tab();

  OPEN rc_kolizje FOR
    SELECT hp.harmonogrampracownikaid
      FROM harmonogrampracownika hp
     WHERE hp.pracownikid = v_pracownikid
       AND p_od < NVL(hp.datazakonczenia, p_do)
       AND p_do > hp.datarozpoczecia;

  LOOP
    FETCH rc_kolizje INTO v_one_id;
    EXIT WHEN rc_kolizje%NOTFOUND;
    v_koliduje.EXTEND;
    v_koliduje(v_koliduje.COUNT) := v_one_id;
  END LOOP;

  CLOSE rc_kolizje;

  IF v_koliduje.COUNT > 0 THEN
    RAISE_APPLICATION_ERROR(-20046, 'Kolizja grafiku: pracownik ma juz zmiane w tym przedziale.');
  END IF;

  INSERT INTO harmonogrampracownika
    (harmonogrampracownikaid, pracownikid, typzmiany, datarozpoczecia, datazakonczenia, zadanie)
  VALUES
    ((SELECT NVL(MAX(harmonogrampracownikaid),0)+1 FROM harmonogrampracownika),
     v_pracownikid, p_typzmiany, p_od, p_do, p_zadanie);

EXCEPTION
  WHEN OTHERS THEN
    IF SQLCODE BETWEEN -20999 AND -20000 THEN
      RAISE;
    END IF;

    IF rc_kolizje%ISOPEN THEN
      CLOSE rc_kolizje;
    END IF;

    RAISE_APPLICATION_ERROR(-20099, 'Blad dodaj_pracownika_do_grafiku: ' || SQLERRM);
```

Procedura dodaj_pracownika_do_grafiku sluzy do dodania pracownika do grafiku pracy na okreslony przedzial czasu. Na poczatku wykonywane sa podstawowe walidacje danych wejsciowych, obejmujace sprawdzenie poprawnosci identyfikatora pracownika oraz logiki dat rozpoczecia i zakonczenia zmiany. W przypadku blednych danych procedura przerywa wykonanie i zwraca komunikat bledu.

Nastepnie sprawdzane jest istnienie pracownika w bazie danych, aby zapobiec dodaniu wpisu dla nieistniejacego rekordu. Kolejnym krokiem jest wykrycie kolizji grafiku z wykorzystaniem kursora referencyjnego (REF CURSOR), ktory pobiera kolidujace wpisy harmonogramu. Dane z kursora sa odczytywane w petli i zapisywane do kolekcji, co pozwala sprawdzic, czy wystepuja konflikty czasowe.

Jesli wykryta zostanie kolizja, procedura zglasza blad i nie wykonuje operacji INSERT. W przeciwnym przypadku nowy wpis zostaje dodany do tabeli harmonogramu pracownika. Na koncu procedura zawiera obsluge bledow, ktora zapewnia bezpieczne przerwanie wykonania oraz czytelne komunikaty bledu w przypadku wystapienia problemow.



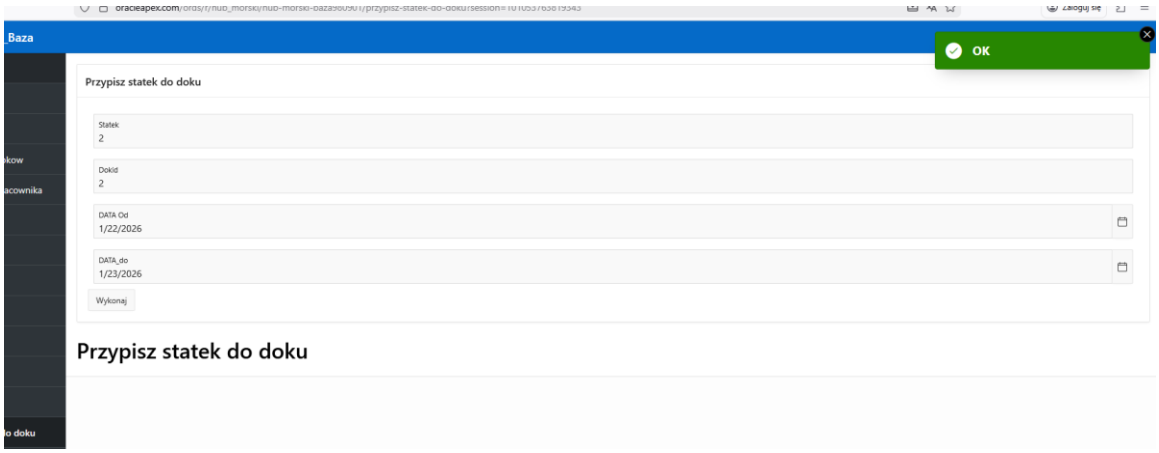
PROCEDURA PRZYPISZ_STATEK_DO_DOKU

```
create or replace PROCEDURE przypisz_statek_do_doku (
  p_statekid_txt IN VARCHAR2,
  p_dokid_txt    IN VARCHAR2,
  p_od           IN DATE,
  p_do           IN DATE
) IS
  -- KOLEKCJA błędów
  v_err SYS.ODCIVARCHAR2LIST := SYS.ODCIVARCHAR2LIST();

  -- OBIEKT
  v_o   t_przypisz_obj;

  v_statekid NUMBER;
  v_dokid    NUMBER;

  -- KURSORY
  CURSOR c_statek(p_id NUMBER) IS SELECT 1 FROM statek WHERE statekid = p_id;
  CURSOR c_dok(p_id NUMBER)    IS SELECT 1 FROM dok    WHERE dokid    = p_id;

  v_dummy NUMBER;

  PROCEDURE add_err(p_msg VARCHAR2) IS
  BEGIN
    v_err.EXTEND;
    v_err(v_err.COUNT) := p_msg;
  END;

  FUNCTION err_text RETURN VARCHAR2 IS
    v_txt VARCHAR2(4000);
  BEGIN
    FOR i IN 1..v_err.COUNT LOOP
      v_txt := v_txt || CASE WHEN i>1 THEN ' | ' END || v_err(i);
    END LOOP;
    RETURN v_txt;
  END;

BEGIN
  -- 1) Walidacje ręcznych wpisów
  IF p_statekid_txt IS NULL THEN
    add_err('STATEKID jest wymagany.');
  ELSIF NOT REGEXP_LIKE(TRIM(p_statekid_txt), '^\d+$') THEN
    add_err('STATEKID musi być liczbą.');
  ELSE
    v_statekid := TO_NUMBER(TRIM(p_statekid_txt));
  END IF;

  IF p_dokid_txt IS NULL THEN
    add_err('DOKID jest wymagany.');
```

```oracle_plsql
ELSIF NOT REGEXP_LIKE(TRIM(p_dokid_txt), '^\d+$') THEN
  add_err('DOKID musi być liczbą.');
ELSE
  v_dokid := TO_NUMBER(TRIM(p_dokid_txt));
END IF;

-- 2) Walidacja dat
IF p_od IS NULL OR p_do IS NULL THEN
  add_err('DATA_OD i DATA_DO są wymagane.');
ELSIF p_do <= p_od THEN
  add_err('DATA_DO musi być późniejsza niż DATA_OD.');
END IF;

IF v_err.COUNT > 0 THEN
  RAISE_APPLICATION_ERROR(-20010, err_text);
END IF;

-- 3) Sprawdzenie istnienia statku i doku przez KURSORY
OPEN c_statek(v_statekid);
FETCH c_statek INTO v_dummy;
IF c_statek%NOTFOUND THEN
  CLOSE c_statek;
  RAISE_APPLICATION_ERROR(-20011, 'Nie istnieje taki STATEKID.');
END IF;
CLOSE c_statek;

OPEN c_dok(v_dokid);
FETCH c_dok INTO v_dummy;
IF c_dok%NOTFOUND THEN
  CLOSE c_dok;
  RAISE_APPLICATION_ERROR(-20012, 'Nie istnieje taki DOKID.');
END IF;
CLOSE c_dok;
```

```oracle_plsql
  -- 4) OBIEKT
  v_o := t_przypisz_obj(v_statekid, v_dokid, p_od, p_do);

  -- 5) INSERT
  INSERT INTO harmonogramdokow
    (harmonogramdokowid, dokid, datarozpoczecia, datazakonczenia, statekid)
  VALUES
    ((SELECT NVL(MAX(harmonogramdokowid),0)+1 FROM harmonogramdokow),
     v_o.dokid, v_o.od_data, v_o.do_data, v_o.statekid);

EXCEPTION
  WHEN OTHERS THEN

    IF SQLCODE BETWEEN -20999 AND -20000 THEN
      RAISE;
    END IF;
    RAISE_APPLICATION_ERROR(-20099, 'Błąd przypisz_statek_do_doku: ' || SQLERRM);
END;
/
```

Procedura przypisz_statek_do_doku służy do przypisania statku do konkretnego doku na określony przedział czasowy. Przyjmuje jako parametry identyfikator statku, identyfikator doku oraz datę rozpoczęcia i zakończenia obsługi. Jej działanie polega na dodaniu odpowiedniego wpisu do harmonogramu obsługi doków, co pozwala na zaplanowanie postoju i obsługi statku w wybranym doku. Procedura wykorzystywana jest w aplikacji Oracle APEX do zarządzania planem obsługi statków i zapewnia spójność danych dotyczących przypisań statków do doków.



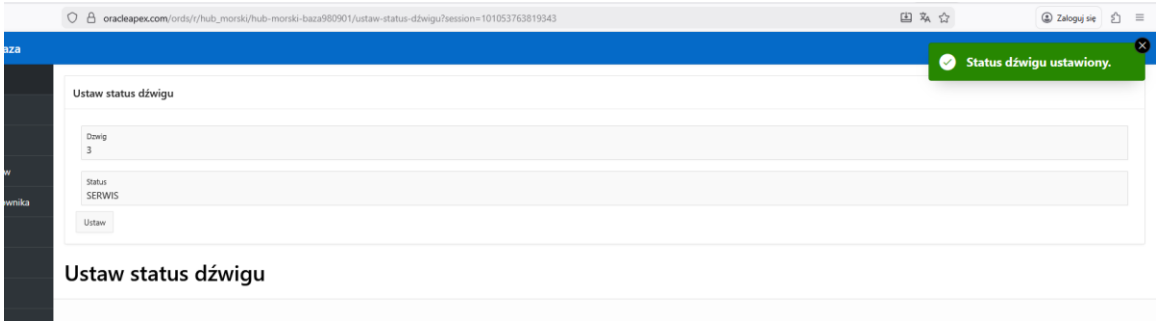
PROCEDURA USTAW_STATUS_DZWIGU

```
create or replace PROCEDURE ustaw_status_dzwigu (
  p_dzwigid_txt IN VARCHAR2,
  p_status_txt  IN VARCHAR2
) IS
  -- KOLEKCJA błędów
  v_err SYS.ODCIVARCHAR2LIST := SYS.ODCIVARCHAR2LIST();

  -- OBIEKT
  v_o   t_status_dzwigu_obj;

  v_dzwigid NUMBER;
  v_status VARCHAR2(50);

  -- KURSOR: sprawdzamy istnienie dźwigu
  CURSOR c_dzwig(p_id NUMBER) IS
    SELECT 1 FROM dzwig WHERE dzwigid = p_id;

  v_dummy NUMBER;

  PROCEDURE add_err(p_msg VARCHAR2) IS
  BEGIN
    v_err.EXTEND;
    v_err(v_err.COUNT) := p_msg;
  END;

  FUNCTION err_text RETURN VARCHAR2 IS
    v_txt VARCHAR2(4000);
  BEGIN
    FOR i IN 1..v_err.COUNT LOOP
      v_txt := v_txt || CASE WHEN i>1 THEN ' | ' END || v_err(i);
    END LOOP;
    RETURN v_txt;
  END;
```

```oracle_plsql
BEGIN
  -- 1) Walidacja ID
  IF p_dzwigid_txt IS NULL THEN
    add_err('DZWIGID jest wymagany.');
  ELSIF NOT REGEXP_LIKE(TRIM(p_dzwigid_txt), '^\d+$') THEN
    add_err('DZWIGID musi być liczbą.');
  ELSE
    v_dzwigid := TO_NUMBER(TRIM(p_dzwigid_txt));
  END IF;

  -- 2) Walidacja statusu
  v_status := UPPER(TRIM(NVL(p_status_txt,'')));

  IF v_status IS NULL OR v_status = '' THEN
    add_err('STATUS jest wymagany.');
  ELSIF v_status NOT IN ('OPERACYJNY','NIEOPERACYJNY','SERWIS','AWARIA') THEN
    add_err('Zły STATUS. Dozwolone: OPERACYJNY/NIEOPERACYJNY/SERWIS/AWARIA.');
  END IF;

  IF v_err.COUNT > 0 THEN
    RAISE_APPLICATION_ERROR(-20030, err_text);
  END IF;

  -- 3) Sprawdzenie istnienia dźwigu przez KURSOR
  OPEN c_dzwig(v_dzwigid);
  FETCH c_dzwig INTO v_dummy;
  IF c_dzwig%NOTFOUND THEN
    CLOSE c_dzwig;
    RAISE_APPLICATION_ERROR(-20031, 'Nie istnieje taki dźwig.');
  END IF;
  CLOSE c_dzwig;
```

```oracle_plsql
  -- 4) OBIEKT
  v_o := t_status_dzwigu_obj(v_dzwigid, v_status);

  -- 5) UPDATE
  UPDATE dzwig
     SET status = v_o.status
   WHERE dzwigid = v_o.dzwigid;

EXCEPTION
  WHEN OTHERS THEN

    IF SQLCODE BETWEEN -20999 AND -20000 THEN
      RAISE;
    END IF;
    RAISE_APPLICATION_ERROR(-20099, 'Błąd ustaw_status_dzwigu: ' || SQLERRM);
END;
/
```

Procedura ustaw_status_dzwigu służy do zmiany statusu dźwigu wykorzystywanego w obsłudze doku. Przyjmuje jako parametry identyfikator dźwigu oraz nowy status, który ma zostać ustawiony. Jej działanie polega na aktualizacji informacji o stanie technicznym dźwigu w bazie danych, co pozwala na bieżące zarządzanie dostępnością sprzętu portowego. Procedura jest wykorzystywana w aplikacji Oracle APEX do kontrolowania stanu infrastruktury technicznej wykorzystywanej przy obsłudze doków.



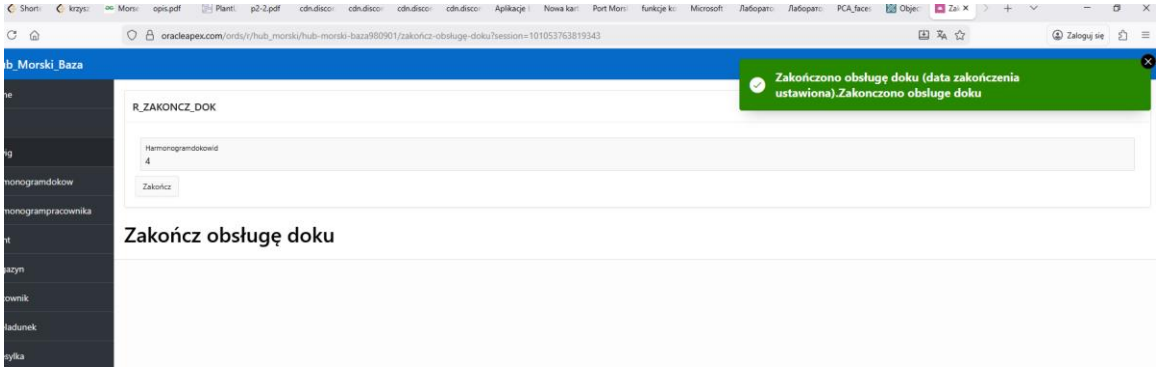
PROCEDURA ZAKONCZ_OBSLUGE_DOKU

```oracle_plsql
create or replace PROCEDURE zakoncz_obsluge_doku (
  p_harmonogramdokowid_txt IN VARCHAR2
) IS

  v_err SYS.ODCIVARCHAR2LIST := SYS.ODCIVARCHAR2LIST();

  -- OBIEKT
  v_o   t_zakoncz_dok_obj;

  v_id  NUMBER;

  -- KURSOR: sprawdzamy istnienie harmonogramu
  CURSOR c_hd(p_id NUMBER) IS
    SELECT 1 FROM harmonogramdokow WHERE harmonogramdokowid = p_id;

  v_dummy NUMBER;

  PROCEDURE add_err(p_msg VARCHAR2) IS
  BEGIN
    v_err.EXTEND;
    v_err(v_err.COUNT) := p_msg;
  END;

  FUNCTION err_text RETURN VARCHAR2 IS
    v_txt VARCHAR2(4000);
  BEGIN
    FOR i IN 1..v_err.COUNT LOOP
      v_txt := v_txt || CASE WHEN i>1 THEN ' | ' END || v_err(i);
    END LOOP;
    RETURN v_txt;
  END;
```

```oracle_plsql
BEGIN
  -- 1) Walidacja ręcznego wpisu ID
  IF p_harmonogramdokowid_txt IS NULL THEN
    add_err('HARMONOGRAMDOKOWID jest wymagany.');
  ELSIF NOT REGEXP_LIKE(TRIM(p_harmonogramdokowid_txt), '^\d+$') THEN
    add_err('HARMONOGRAMDOKOWID musi być liczbą.');
  ELSE
    v_id := TO_NUMBER(TRIM(p_harmonogramdokowid_txt));
  END IF;

  IF v_err.COUNT > 0 THEN
    RAISE_APPLICATION_ERROR(-20020, err_text);
  END IF;

  -- 2) Sprawdzenie istnienia przez KURSOR
  OPEN c_hd(v_id);
  FETCH c_hd INTO v_dummy;
  IF c_hd%NOTFOUND THEN
    CLOSE c_hd;
    RAISE_APPLICATION_ERROR(-20021, 'Nie istnieje taki harmonogram doku.');
  END IF;
  CLOSE c_hd;

  -- 3) OBIEKT
  v_o := t_zakoncz_dok_obj(v_id);

  -- 4) UPDATE
  UPDATE harmonogramdokow
     SET datazakonczenia = SYSDATE
   WHERE harmonogramdokowid = v_o.harmonogramdokowid;
```

```oracle_plsql
EXCEPTION
  WHEN OTHERS THEN
    IF SQLCODE BETWEEN -20999 AND -20000 THEN
      RAISE;
    END IF;
    RAISE_APPLICATION_ERROR(-20099, 'Błąd zakoncz_obsluge_doku: ' || SQLERRM);
END;
/
```

Procedura zakoncz_obsluge_doku służy do zakończenia obsługi doku poprzez aktualizację istniejącego wpisu w harmonogramie. Jako parametr przyjmuje identyfikator rekordu harmonogramu obsługi doku, a jej wykonanie polega na ustawieniu daty zakończenia obsługi. Dzięki tej procedurze możliwe jest jednoznaczne zamknięcie trwającej obsługi doku, co zapobiega dalszemu traktowaniu jej jako aktywnej i możliwiaplanowanie kolejnych operacji.

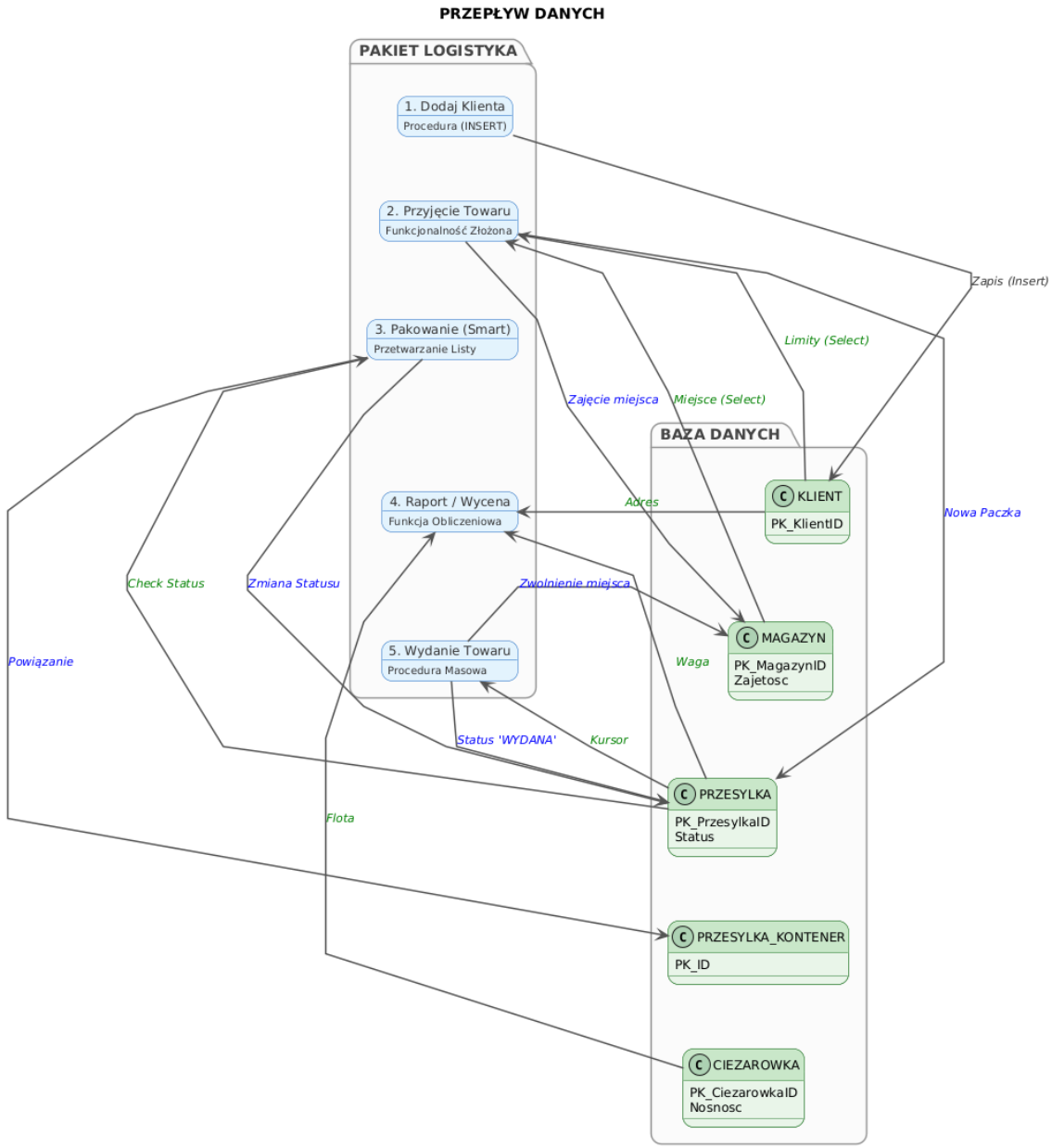


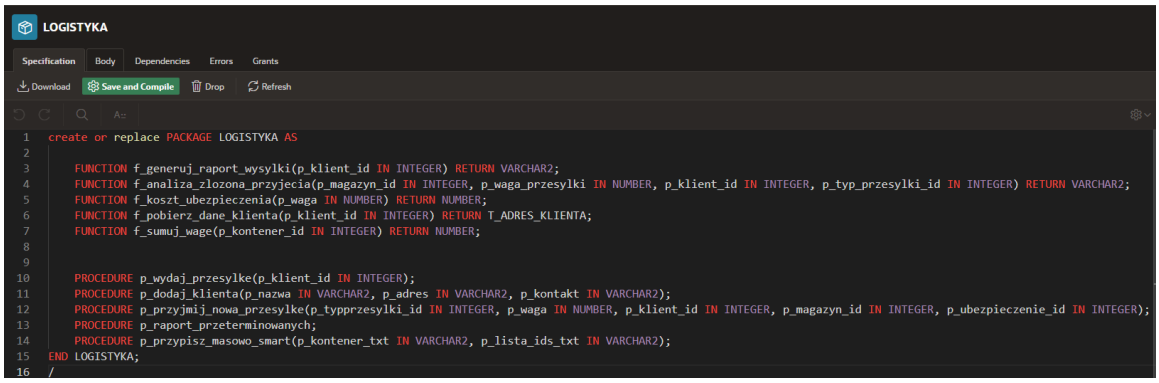
3.2. Pakiet Logistyka

Pakiet LOGISTYKA został zaprojektowany w celu centralizacji logiki biznesowej związanej z kompleksową obsługą łańcucha dostaw w hubie morskim. Jego głównym zadaniem jest zapewnienie spójnych zasad walidacji przyjęć towarowych, automatyzacja procesów magazynowych oraz optymalizacja planowania transportu, tak aby wszystkie operacje na przesyłkach i klientach były realizowane w jednym, kontrolowanym miejscu w bazie danych.

W pierwszej części pakietu znajdują się funkcje analityczne i obliczeniowe, których celem jest weryfikacja warunków oraz dostarczanie przetworzonych danych. Funkcja `f_pobierz_dane_klienta` odpowiada za pobieranie strukturalnych informacji o kontrahencie w formie obiektowej, co zapewnia spójność danych adresowych w całym systemie. Funkcja `f_analiza_zlozona_przyjecia` pełni rolę kluczowego mechanizmu walidacyjnego – przed fizycznym przyjęciem towaru sprawdza dostępną pojemność wagową magazynu oraz weryfikuje indywidualne limity ilościowe przypisane do klienta, zapobiegając przepełnieniu stref składowania. Funkcje pomocnicze, takie jak `f_koszt_ubezpieczenia` oraz `f_sumuj_wage`, wykonują niezbędne kalkulacje finansowe i techniczne. Funkcja `f_generuj_raport_wysylki` pełni rolę funkcji nadrzędnej dla procesu wyceny – symuluje logistykę wyjścia, obliczając zapotrzebowanie na flotę transportową oraz estymowane koszty trasy dla wskazanego klienta.

Druga część pakietu zawiera procedury operacyjne, które odpowiadają za faktyczne modyfikowanie danych w systemie. Procedura `p_dodaj_klienta` realizuje proces rejestracji nowego kontrahenta, dbając o poprawność danych wejściowych i unikalność wpisów. Procedura `p_przyjmij_nowa_przesylke` obsługuje proces wejścia towaru, dokonując wpisu w rejestrze oraz aktualizując zajętość magazynu, pod warunkiem pomyślnej walidacji przez funkcje analityczne. Procedura `p_przypisz_masowo_smart` usprawnia logistykę wewnętrzną, umożliwiając grupowe przypisywanie przesyłek do kontenerów na podstawie listy identyfikatorów. Procedura `p_wydaj_przesylke` zamyka cykl życia towaru w magazynie – wykorzystując mechanizm kursorów, masowo aktualizuje statusy paczek na „WYDANA” i zwalnia zasoby magazynowe. Dodatkowo procedura `p_raport_przeterminowanych` pełni funkcję utrzymaniową, monitorując i oznaczając towary zalegające powyżej ustalonego czasu.






```
create or replace PACKAGE LOGISTYKA AS

    FUNCTION f_generuj_raport_wysylki(p_klient_id IN INTEGER) RETURN VARCHAR2;
    FUNCTION f_analiza_zlozona_przyjecia(p_magazyn_id IN INTEGER, p_waga_przesylki IN NUMBER, p_klient_id IN INTEGER, p_typ_przesylki_id IN INTEGER) RETURN VARCHAR2;
    FUNCTION f_koszt_ubezpieczenia(p_waga IN NUMBER) RETURN NUMBER;
    FUNCTION f_pobierz_dane_klienta(p_klient_id IN INTEGER) RETURN T_ADRES_KLIENTA;
    FUNCTION f_sumuj_wage(p_kontener_id IN INTEGER) RETURN NUMBER;


    PROCEDURE p_wydaj_przesylke(p_klient_id IN INTEGER);
    PROCEDURE p_dodaj_klienta(p_nazwa IN VARCHAR2, p_adres IN VARCHAR2, p_kontakt IN VARCHAR2);
    PROCEDURE p_przyjmij_nowa_przesylke(p_typprzesylki_id IN INTEGER, p_waga IN NUMBER, p_klient_id IN INTEGER, p_magazyn_id IN INTEGER, p_ubezpieczenie_id IN INTEGER);
    PROCEDURE p_raport_przeterminowanych;
    PROCEDURE p_przypisz_masowo_smart(p_kontener_txt IN VARCHAR2, p_lista_ids_txt IN VARCHAR2);
END LOGISTYKA;
/
```

- Funkcje

SF_ANALIZA_PRZYJECIA

Code | Dependencies | Errors | Grants

Download | Save and Compile | Drop | Refresh

```oracle_plsql
create or replace FUNCTION SF_ANALIZA_PRZYJECIA(
    p_magazyn_id       IN INTEGER,
    p_waga_przesylki   IN NUMBER,
    p_klient_id        IN INTEGER,
    p_typ_przesylki_id IN INTEGER
) RETURN VARCHAR2 IS
    v_status VARCHAR2(50);
    v_max    NUMBER;
    v_zajete NUMBER;
    v_ile    NUMBER;
BEGIN
    -- Sprawdzenie czy magazyn istnieje i jaki ma status
    BEGIN
        SELECT status, maksymalnapojemnosc, NVL(zajetaprzestrzen, 0)
        INTO v_status, v_max, v_zajete
        FROM magazyn WHERE magazynid = p_magazyn_id;

        IF v_status IS NULL THEN RETURN 'BLAD_STATUS_MAGAZYNU'; END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN RETURN 'BLAD_NIEZNANY_MAGAZYN';
    END;

    -- Sprawdzenie czy jest miejsce
    IF (v_zajete + p_waga_przesylki) > v_max THEN
        RETURN 'BLAD_BRAK_MIEJSCA';
    END IF;

    -- Sprawdzenie limitu klienta
    SELECT COUNT(*) INTO v_ile FROM przesylka WHERE klientid = p_klient_id AND status = 'PRZYJETA';
    IF v_ile >= 50 THEN RETURN 'BLAD_LIMIT_KLIENTA'; END IF;

    RETURN 'TAK';
END;
/
```

Funkcja sf_analiza _przyjecia służy do wieloetapowej weryfikacji możliwości przyjęcia nowej przesyłki na stan magazynowy. Jako kryterium akceptacji przyjęto spełnienie dwóch warunków jednocześnie: dostępności miejsca w magazynie oraz nieprzekroczenia indywidualnego limitu paczek dla klienta. Funkcja przyjmuje jako parametry identyfikator magazynu, wagę przesyłki oraz identyfikator klienta. Następnie pobiera z tabeli MAGAZYN aktualne obłożenie i porównuje je z maksymalną pojemnością (uwzględniając nową wagę). Równolegle zlicza liczbę aktywnych przesyłek przypisanych do klienta. Jeżeli oba warunki są spełnione, funkcja zwraca wartość 'TAK'. W przeciwnym przypadku zwraca kod błędu, np. 'BLAD_BRAK_MIEJSCA' lub 'BLAD_LIMIT_KLIENTA', co pozwala na precyzyjną obsługę wyjątku w procedurze wywołującej. Funkcja wykorzystywana jest jako kluczowy element logiki biznesowej w procedurze p_przyjmij_nowa_przesylke.

SF_POBIERZ_DANE_KLIENTA

Code Dependencies Errors Grants

Download Save and Compile Drop Refresh

```
create or replace FUNCTION SF_POBIERZ_DANE_KLIENTA(p_klient_id IN INTEGER) RETURN T_ADRES_KLIENTA IS
    v_klient T_ADRES_KLIENTA;
BEGIN
    SELECT T_ADRES_KLIENTA(nazwa, adres, numerkontaktu)
    INTO v_klient
    FROM klient WHERE klientid = p_klient_id;
    RETURN v_klient;
END;
/
```

Funkcja sf_pobierz_dane_klienta służy do hermetyzacji procesu pobierania danych teleadresowych kontrahenta. Jej zadaniem jest zwrócenie kompletnego obiektu typu T_ADRES_KLIENTA na podstawie podanego identyfikatora. Funkcja przyjmuje jako parametr ID klienta, a następnie wykonuje zapytanie do tabeli KLIENT, mapując kolumny (nazwa, adres, kontakt) na atrybuty typu obiektowego. Jeżeli klient o podanym identyfikatorze istnieje, zwracana jest zainicjalizowana instancja obiektu. W sytuacji braku danych funkcja może zwrócić wartość NULL, co jest sygnałem dla aplikacji o błędnym identyfikatorze. Funkcja wykorzystywana jest głównie w modułach raportowych, w tym wewnątrz funkcji f_generuj_raport_wysylki.

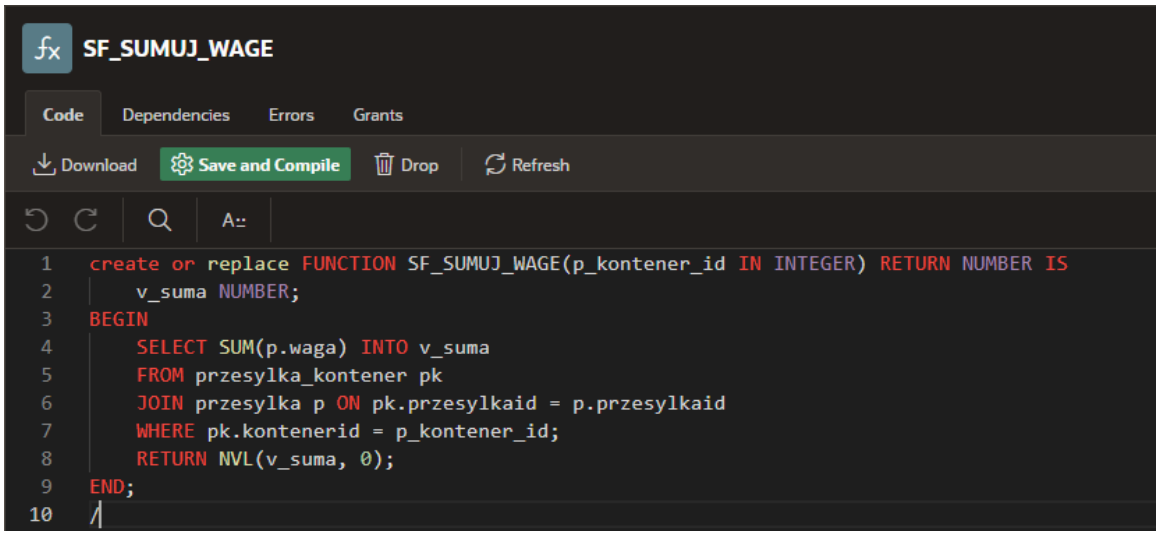
SF_KOSZT_UBEZPIECZENIA

Code Dependencies Errors Grants

Download Save and Compile Drop Refresh

```
create or replace FUNCTION SF_KOSZT_UBEZPIECZENIA(p_waga IN NUMBER) RETURN NUMBER IS
BEGIN
    RETURN (p_waga / 100) * 50;
END;
/
```

Funkcja f_koszt_ubezpieczenia służy do automatycznej kalkulacji kosztów dodatkowej ochrony towaru w zależności od jego wagi. Jako kryterium wyceny przyjęto progi wagowe zdefiniowane w regułach biznesowych hubu. Funkcja przyjmuje jako parametr wagę przesyłki (liczba). Następnie sprawdza, w jakim przedziale mieści się podana wartość (np. do 50 kg i powyżej) i aplikuje odpowiedni mnożnik lub stałą stawkę. Wynikiem działania funkcji jest kwota ubezpieczenia wyrażona w walucie systemowej. Funkcja ta jest wykorzystywana w procesach wyceny usług oraz podczas generowania podsumowań dla klienta.



Funkcja sf_sumuj_wage służy do agregacji danych wagowych dla wskazanego kontenera logistycznego. Jej celem jest kontrola, czy kontener nie został przeładowany. Funkcja przyjmuje jako parametr identyfikator kontenera, a następnie sumuje wagi wszystkich przesyłek powiązanych z tym kontenerem poprzez tabelę łączącą PRZESYLKA_KONTENER. Jeżeli kontener jest pusty, funkcja zwraca wartość 0, obsługując tym samym przypadki braku danych (NULL) za pomocą funkcji NVL. Wynik funkcji jest kluczowy przy planowaniu załadunku na środki transportu o ograniczonej nośności.

SF_GENERUJ_RAPORT_WYSYLKI

Code | Dependencies | Errors | Grants

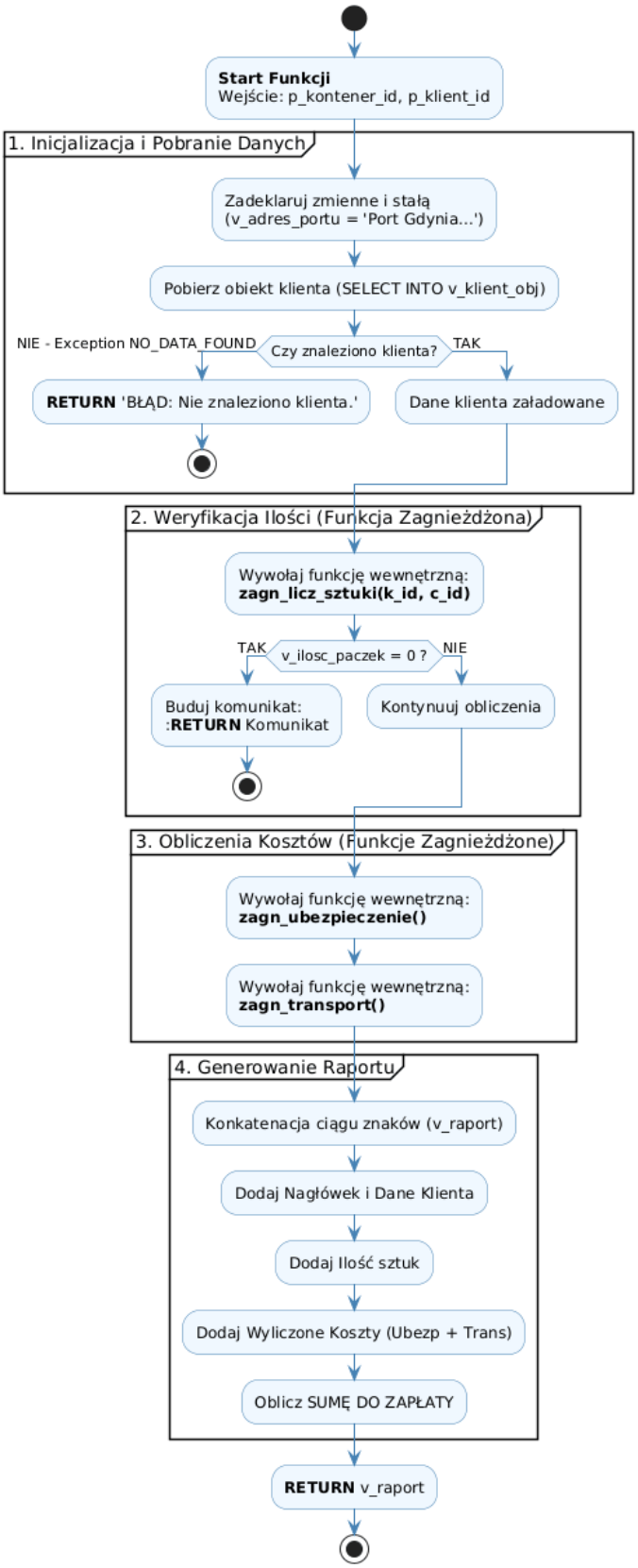
Download | Save and Compile | Drop | Refresh

```oracle_plsql
create or replace FUNCTION SF_GENERUJ_RAPORT_WYSYLKI(
    p_klient_id IN INTEGER
) RETURN VARCHAR2 IS

    -- Zmienne
    v_klient_obj  T_ADRES_KLIENTA;
    v_adres_hubu  VARCHAR2(100) := 'Hub Centralny, ul. Portowa 10';

    v_laczna_waga    NUMBER := 0;
    v_ilosc_paczek   NUMBER := 0;

    -- Zmienne Floty
    v_ladownosc_kg   NUMBER; -- Tutaj przechowamy wartość w KG
    v_typ_auta       VARCHAR2(100);

    v_potrzebne_auta NUMBER;
    v_koszt_trans    NUMBER;
    v_raport         VARCHAR2(4000);

    -- ZAGNIEŻDŻONA 1: Logika Floty
    FUNCTION zagn_licz_auta(p_masa_calkowita NUMBER, p_limit NUMBER) RETURN NUMBER IS
    BEGIN
        IF p_masa_calkowita = 0 THEN RETURN 0; END IF;
        IF p_limit IS NULL OR p_limit = 0 THEN RETURN 999; END IF;
        RETURN CEIL(p_masa_calkowita / p_limit);
    END;

    -- ZAGNIEŻDŻONA 2: Logika Kosztów
    FUNCTION zagn_wycena_trasy(p_adres_k VARCHAR2, p_adres_h VARCHAR2, p_aut NUMBER) RETURN NUMBER IS
        v_odleglosc NUMBER;
        v_stawka_km NUMBER := 5;
    BEGIN
        v_odleglosc := ABS(LENGTH(p_adres_k) - LENGTH(p_adres_h)) * 2;
        IF v_odleglosc < 50 THEN v_odleglosc := 50; END IF;
        RETURN v_odleglosc * v_stawka_km * p_aut;
    END;

BEGIN
```

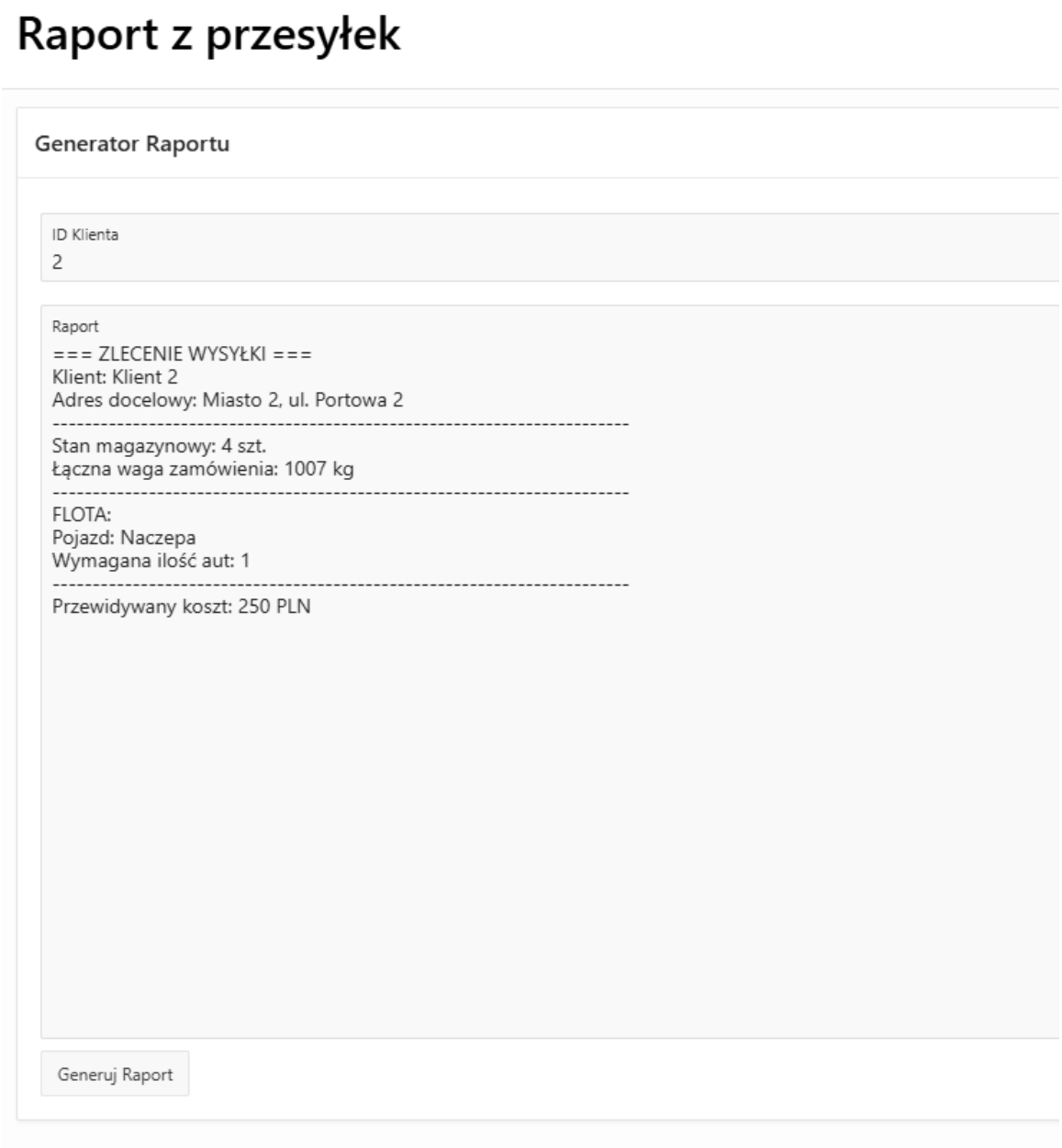
```oracle_plsql
    -- 1. Pobierz Klienta
    BEGIN
        SELECT T_ADRES_KLIENTA(nazwa, adres, numerkontaktu)
        INTO v_klient_obj
        FROM klient WHERE klientid = p_klient_id;
    EXCEPTION WHEN NO_DATA_FOUND THEN
        RETURN 'BŁĄD: Nie znaleziono klienta.';
    END;

    -- 2. Pobierz Ciężarówkę i PRZELICZ TONY NA KG
    BEGIN
        -- MNOŻYMY RAZY 1000, bo w bazie są tony!
        SELECT MAX(nosnosc) * 1000, MAX(typ)
        INTO v_ladownosc_kg, v_typ_auta
        FROM ciezarowka;

        -- Zabezpieczenie (Domyślnie 24 tony = 24000 kg)
        IF v_ladownosc_kg IS NULL THEN
            v_ladownosc_kg := 24000;
            v_typ_auta := 'Standard (Brak aut w bazie)';
        END IF;
    EXCEPTION WHEN OTHERS THEN
        v_ladownosc_kg := 24000;
        v_typ_auta := 'Błąd pobrania floty';
    END;

    -- 3. Policz towar
    SELECT NVL(SUM(waga), 0), COUNT(*)
    INTO v_laczna_waga, v_ilosc_paczek
    FROM przesylka
    WHERE klientid = p_klient_id AND status = 'PRZYJETA';

    IF v_ilosc_paczek = 0 THEN
        RETURN 'Klient ' || v_klient_obj.nazwa_klienta || ' nie posiada paczek gotowych do wysyłki.';
    END IF;

    -- 4. Logistyka
    v_potrzebne_auta := zagn_licz_auta(v_laczna_waga, v_ladownosc_kg);
```

```oracle_plsql
    -- 4. Logistyka
    v_potrzebne_auta := zagn_licz_auta(v_laczna_waga, v_ladownosc_kg);
    v_koszt_trans    := zagn_wycena_trasy(v_klient_obj.adres_klienta, v_adres_hubu, v_potrzebne_auta);

    -- 5. Generuj Raport
    v_raport := '=== ZLECENIE WYSYŁKI ===' || CHR(10) ||
                'Klient: ' || v_klient_obj.nazwa_klienta || CHR(10) ||
                'Adres docelowy: ' || v_klient_obj.adres_klienta || CHR(10) ||
                '-----------------------------------------------------------------------' || CHR(10) ||
                'Stan magazynowy: ' || v_ilosc_paczek || ' szt.' || CHR(10) ||
                'Łączna waga zamówienia: ' || v_laczna_waga || ' kg' || CHR(10) ||
                '-----------------------------------------------------------------------' || CHR(10) ||
                'FLOTA:' || CHR(10) ||
                'Pojazd: ' || v_typ_auta || CHR(10) ||
                'Wymagana ilość aut: ' || v_potrzebne_auta || CHR(10) ||
                '-----------------------------------------------------------------------' || CHR(10) ||
                'Przewidywany koszt: ' || v_koszt_trans || ' PLN';

    RETURN v_raport;
END;
/
```

Algorytm Funkcji: SF_GENERUJ_RAPORT_ZAMOWIENIA



Funkcja f_generuj_raport_wysylki służy do symulacji procesu logistycznego i przygotowania wstępnej oferty transportowej. Jest to funkcja nadrzędna ("super-funkcja"), która agreguje logikę z innych modułów. Przyjmuje identyfikator klienta, a następnie oblicza łączną wagę towarów gotowych do wysyłki, dobiera odpowiednią flotę (liczbę ciężarówek) i estymuje koszt transportu na podstawie odległości (symulowanej na bazie adresu). Wynikiem działania funkcji jest sformatowany ciąg znaków (String) zawierający czytelne podsumowanie dla użytkownika końcowego.



- Procedury

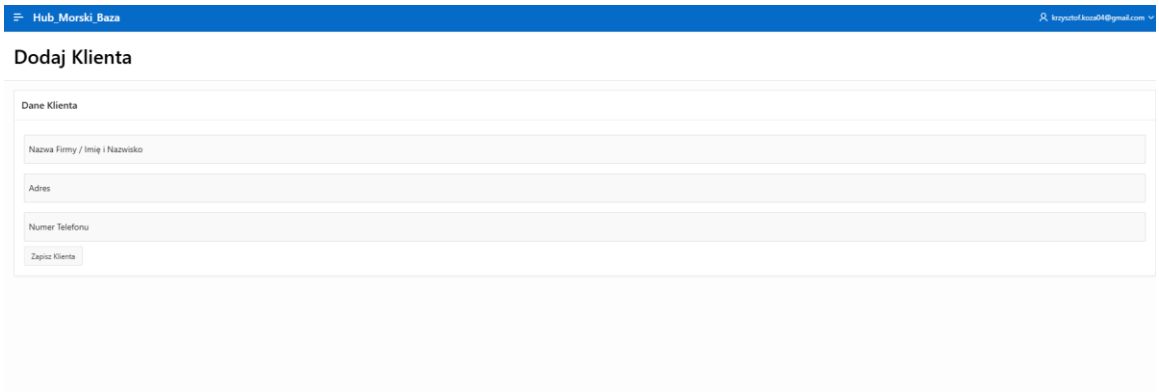
SP_DODAJ_KLIENTA

Code | Dependencies | Errors | Grants

Download | Save and Compile | Drop | Refresh

```
create or replace PROCEDURE SP_DODAJ_KLIENTA(
    p_nazwa   IN VARCHAR2,
    p_adres   IN VARCHAR2,
    p_kontakt IN VARCHAR2
) IS
    v_new_id INTEGER;
BEGIN
    -- 1. Walidacja (Sprawdzamy czy wpisano dane)
    IF p_nazwa IS NULL THEN
        RAISE_APPLICATION_ERROR(-20030, 'Błąd: Nazwa klienta jest wymagana!');
    END IF;

    IF p_adres IS NULL THEN
        RAISE_APPLICATION_ERROR(-20031, 'Błąd: Adres klienta jest wymagany!');
    END IF;

    -- 2. Generowanie nowego ID (MAX + 1)
    SELECT NVL(MAX(klientid), 0) + 1 INTO v_new_id FROM klient;

    -- 3. Wstawienie do tabeli
    INSERT INTO klient (klientid, nazwa, adres, numerkontaktu)
    VALUES (v_new_id, p_nazwa, p_adres, p_kontakt);

    COMMIT;
END;
/
```

Procedura sp_dodaj_klienta służy do rejestracji nowego kontrahenta w systemie, zapewniając integralność wprowadzanych danych. Przyjmuje jako parametry nazwę, adres oraz dane kontaktowe. Przed wykonaniem operacji INSERT, procedura dokonuje walidacji danych wejściowych – w szczególności sprawdza, czy pole nazwy nie jest puste. W przypadku wykrycia błędnych danych (NULL), procedura podnosi wyjątek aplikacyjny (RAISE_APPLICATION_ERROR), przerywając transakcję i zwracając komunikat do użytkownika. Procedura automatycznie generuje unikalny identyfikator klienta (sekwencja lub MAX+1), co zwalnia użytkownika z konieczności ręcznego zarządzania kluczami głównymi.



SP_PRZYJMIJ_PRZESYLKE

Code | Dependencies | Errors | Grants

Download | Save and Compile | Drop | Refresh

```
create or replace PROCEDURE SP_PRZYJMIJ_PRZESYLKE(
    p_typprzesylki_id IN INTEGER,
    p_waga            IN NUMBER,
    p_klient_id       IN INTEGER,
    p_magazyn_id      IN INTEGER,
    p_ubezpieczenie_id IN INTEGER
) IS
    v_chk VARCHAR2(100);
    v_id  INTEGER;
BEGIN
    -- Wywołanie funkcji wolnostojącej
    v_chk := SF_ANALIZA_PRZYJECIA(p_magazyn_id, p_waga, p_klient_id, p_typprzesylki_id);

    IF v_chk <> 'TAK' THEN
        RAISE_APPLICATION_ERROR(-20005, 'Odmowa przyjęcia: ' || v_chk);
    END IF;

    SELECT NVL(MAX(przesylkaid), 0) + 1 INTO v_id FROM przesylka;

    INSERT INTO przesylka (przesylkaid, typprzesylkiid, waga, klientid, magazynid, dataprzyjecia, status, ubezpieczenieid, kontener_kontenerid, przeladunek_przeladunekid)
    VALUES (v_id, p_typprzesylki_id, p_waga, p_klient_id, p_magazyn_id, SYSDATE, 'PRZYJETA', p_ubezpieczenie_id, 0, 0);

    UPDATE magazyn SET zajetaprzestrzen = NVL(zajetaprzestrzen, 0) + p_waga WHERE magazynid = p_magazyn_id;
    COMMIT;
END;
/
```

Procedura sp_przyjmij _przesylke służy do sformalizowania procesu wejścia towaru na magazyn. Procedura przyjmuje parametry opisujące przesyłkę (typ, waga, właściciel, magazyn docelowy). W pierwszym kroku wywołuje funkcję logiczną f_analiza_zlozona_przyjecia. Jeżeli funkcja ta zwróci wynik pozytywny ('TAK'), procedura dokonuje wstawienia rekordu do tabeli PRZESYLKA ze statusem 'PRZYJETA' oraz natychmiast aktualizuje (zwiększa) wartość zajętej przestrzeni w tabeli MAGAZYN. Dzięki temu stan magazynowy jest zawsze aktualny (konsystentny) w ramach jednej transakcji bazodanowej.

Przyjęcie Towaru

Formularz Przyjęcia

ID Klienta

ID Magazynu

ID Typu Przesyłki

ID Ubezpieczenia

Waga (kg)

Zapisz

SP_PAKUJ_KONTENER_SMART

Code | Dependencies | Errors | Grants

Download | Save and Compile | Drop | Refresh

```oracle_plsql
create or replace PROCEDURE SP_PAKUJ_KONTENER_SMART(
    p_kontener_txt  IN VARCHAR2,
    p_lista_ids_txt IN VARCHAR2
) IS
    v_err SYS.ODCIVARCHAR2LIST := SYS.ODCIVARCHAR2LIST();
    v_kontener_id NUMBER;
    v_lista_robocza APEX_T_VARCHAR2;
    v_pojedyncze_id NUMBER;
    v_nowe_pk_id NUMBER;

    -- Zmienna do sprawdzenia statusu
    v_aktualny_status VARCHAR2(50);

    CURSOR c_kontener(p_id NUMBER) IS SELECT 1 FROM kontener WHERE kontenerid = p_id;
    v_dummy NUMBER;

    PROCEDURE add_err(msg VARCHAR2) IS BEGIN v_err.EXTEND; v_err(v_err.COUNT) := msg; END;
BEGIN
    -- 1. Walidacje Wstępne (Tak jak było)
    IF p_kontener_txt IS NULL THEN add_err('ID Kontenera jest wymagane.');
    ELSIF NOT REGEXP_LIKE(p_kontener_txt, '^[0-9]+$') THEN add_err('ID Kontenera musi być liczbą.');
    ELSE v_kontener_id := TO_NUMBER(p_kontener_txt); END IF;

    IF p_lista_ids_txt IS NULL THEN add_err('Lista przesyłek jest wymagana.');
    ELSIF NOT REGEXP_LIKE(p_lista_ids_txt, '^[0-9, ]+$') THEN add_err('Lista może zawierać tylko cyfry i przecinki.');
    END IF;

    IF v_err.COUNT > 0 THEN RAISE_APPLICATION_ERROR(-20010, 'Błędy walidacji: ' || v_err(1)); END IF;

    OPEN c_kontener(v_kontener_id);
    FETCH c_kontener INTO v_dummy;
    IF c_kontener%NOTFOUND THEN
        CLOSE c_kontener; RAISE_APPLICATION_ERROR(-20011, 'Podany Kontener nie istnieje.');
    END IF;
    CLOSE c_kontener;

    -- 2. Przetwarzanie listy (Z POPRAWKĄ)
    v_lista_robocza := APEX_STRING.SPLIT(p_lista_ids_txt, ',');

    FOR i IN 1..v_lista_robocza.COUNT LOOP
        BEGIN
            v_pojedyncze_id := TO_NUMBER(TRIM(v_lista_robocza(i)));

            -- A. SPRAWDZAMY STATUS (To jest ta nowość!)
            SELECT status INTO v_aktualny_status
            FROM przesylka
            WHERE przesylkaid = v_pojedyncze_id;

            -- B. JEŚLI STATUS JEST ZŁY -> KRZYCZYMY BŁĘDEM
            IF v_aktualny_status = 'WYDANA' THEN
                RAISE_APPLICATION_ERROR(-20020, 'Błąd: Przesyłka ID ' || v_pojedyncze_id || ' została już WYDANA z magazynu!');
            ELSIF v_aktualny_status = 'W_KONTENERZE' THEN
                RAISE_APPLICATION_ERROR(-20021, 'Błąd: Przesyłka ID ' || v_pojedyncze_id || ' jest już zapakowana!');
            ELSIF v_aktualny_status = 'PRZETERMINOWANA' THEN
                RAISE_APPLICATION_ERROR(-20022, 'Błąd: Przesyłka ID ' || v_pojedyncze_id || ' jest przeterminowana! Utylizuj zamiast pakować.');
            END IF;

            -- C. JEŚLI WSZYSTKO OK -> PAKUJEMY
            SELECT NVL(MAX(id), 0) + 1 INTO v_nowe_pk_id FROM przesylka_kontener;
            INSERT INTO przesylka_kontener (id, przesylkaid, kontenerid, datadodania)
            VALUES (v_nowe_pk_id, v_pojedyncze_id, v_kontener_id, SYSDATE);

            UPDATE przesylka SET status = 'W_KONTENERZE' WHERE przesylkaid = v_pojedyncze_id;

        EXCEPTION
            WHEN NO_DATA_FOUND THEN
                -- Jeśli wpisałeś ID, którego nie ma w bazie
                RAISE_APPLICATION_ERROR(-20023, 'Błąd: Przesyłka o ID ' || v_pojedyncze_id || ' nie istnieje!');
            WHEN OTHERS THEN
                -- Jeśli to nasz błąd logiczny (powyżej), puszczamy go dalej
                RAISE;
        END;
    END LOOP;
    COMMIT;
END;
/
```

Procedura sp_przypisz_masowo_smart służy do optymalizacji procesu pakowania, umożliwiając obsługę wielu przesyłek w jednej operacji. Przyjmuje identyfikator kontenera oraz listę identyfikatorów przesyłek w formie ciągu tekstowego (np. '1,2,3'). Wewnątrz procedury zaimplementowano logikę parsowania ciągu znaków, która rozbija listę na pojedyncze ID. Następnie w pętli dla każdego identyfikatora następuje weryfikacja statusu przesyłki i – jeśli jest poprawny – przypisanie jej do kontenera w tabeli łączącej oraz zmiana statusu na 'W_KONTENERZE'. Mechanizm ten znacznie przyspiesza pracę magazyniera, eliminując konieczność pojedynczego "klikania" każdej paczki.

Pakowanie Kontenera

Lista

Przesyłkaid	Typprzesyłkiid	Waga	Wymiary	Klientid	Magazynid	Dataprzyjecia	Datawydania	Status	Ubezpieczenieid	Kontener Kontenerid	Przeladunek Przeladunekid
1	1	110	Wymiary_1	1	1	1/3/2025		W_KONTENERZE	1	1	1
2	2	120	Wymiary_2	2	2	1/5/2025	1/12/2026	W_KONTENERZE	2	2	2
3	3	130	Wymiary_3	3	3	1/7/2025		W_KONTENERZE	3	3	3
4	4	140	Wymiary_4	4	4	1/9/2025		W_DRODZE	4	4	4
5	5	150	Wymiary_5	5	5	1/11/2025		PRZETERMINOWANA	5	5	5
6	6	160	Wymiary_6	6	6	1/13/2025		W_KONTENERZE	6	6	6
7	7	170	Wymiary_7	7	7	1/15/2025		PRZETERMINOWANA	7	7	7
8	8	180	Wymiary_8	8	8	1/17/2025		W_DRODZE	8	8	8
9	9	190	Wymiary_9	9	9	1/19/2025		PRZETERMINOWANA	9	9	9
10	10	200	Wymiary_10	10	10	1/21/2025		W_DRODZE	10	10	10
11	3	45.7		2	1	1/12/2026		W_KONTENERZE	2	0	0
12	1	350		3	4	1/12/2026		W_KONTENERZE	1	0	0
13	2	200		9	2	1/12/2026		W_KONTENERZE	5	0	0
14	3	175.6		3	1	1/12/2026		W_KONTENERZE	2	0	0
15	1	240.2		2	3	1/13/2026	1/13/2026	W_KONTENERZE	4	0	0

1 - 15 Next

Pakowanie do kontenera

Numer Kontenera

Lista Paczek (np. 1,2,3)

Spakuj do kontenera

SP_WYDAJ_PRZESYLKE

Code | Dependencies | Errors | Grants

Download | Save and Compile | Drop | Refresh

```
create or replace PROCEDURE SP_WYDAJ_PRZESYLKE(p_przesylka_id IN INTEGER) IS
    v_w NUMBER;
    v_m INTEGER;
BEGIN
    SELECT waga, magazynid INTO v_w, v_m FROM przesylka WHERE przesylkaid = p_przesylka_id;

    UPDATE przesylka SET datawydania = SYSDATE, status = 'WYDANA' WHERE przesylkaid = p_przesylka_id;
    UPDATE magazyn SET zajetaprzestrzen = zajetaprzestrzen - v_w WHERE magazynid = v_m;
    COMMIT;
EXCEPTION
    WHEN NO_DATA_FOUND THEN RAISE_APPLICATION_ERROR(-20003, 'Brak przesyłki o podanym ID');
END;
/
```

Procedura sp_wydaj_przesylke służy do masowego wydania towaru z magazynu dla wskazanego klienta, co kończy cykl życia przesyłki w obrębie hubu. Wykorzystuje mechanizm kursora jawnego (CURSOR), aby pobrać wszystkie przesyłki klienta o statusie 'PRZYJETA'. W pętli, dla każdej znalezionej przesyłki, procedura wykonuje dwie operacje: zmienia jej status na 'WYDANA' (w tabeli PRZESYLKA) oraz pomniejsza zajętość odpowiedniego magazynu o wagę wydanej paczki (w tabeli MAGAZYN). Zastosowanie transakcyjności gwarantuje, że zwolnienie miejsca w magazynie nastąpi tylko wtedy, gdy towar faktycznie zmieni status.

Wydanie Towaru

Szukaj Klienta

Podaj ID Klienta
2

Pokaż Paczki

Paczki do wydania

Lista

Id Paczki	Waga (kg)	Data Przyjęcia	Obecny Status	Id Magazynu
27	450	1/13/2026	PRZYJETA	1
30	189	1/13/2026	PRZYJETA	1
31	23	1/13/2026	PRZYJETA	1
32	345	1/13/2026	PRZYJETA	1

1 - 4

Wydanie

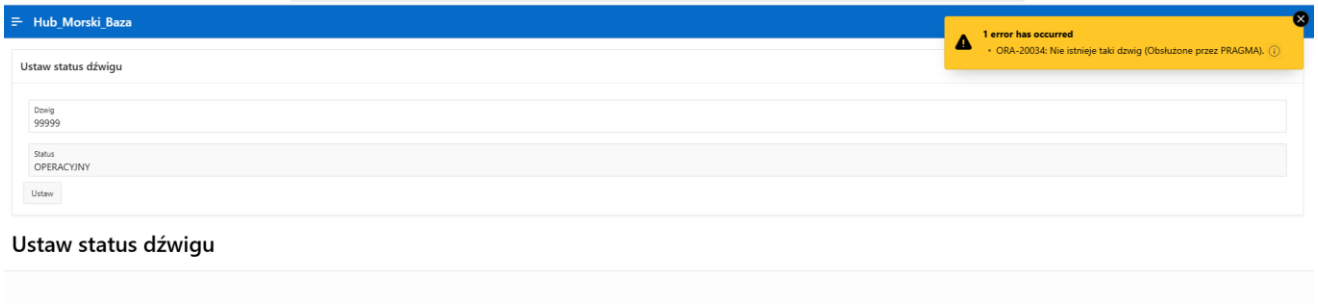
Wpisz ID klienta

Wydaj

4. Obsługa błędów

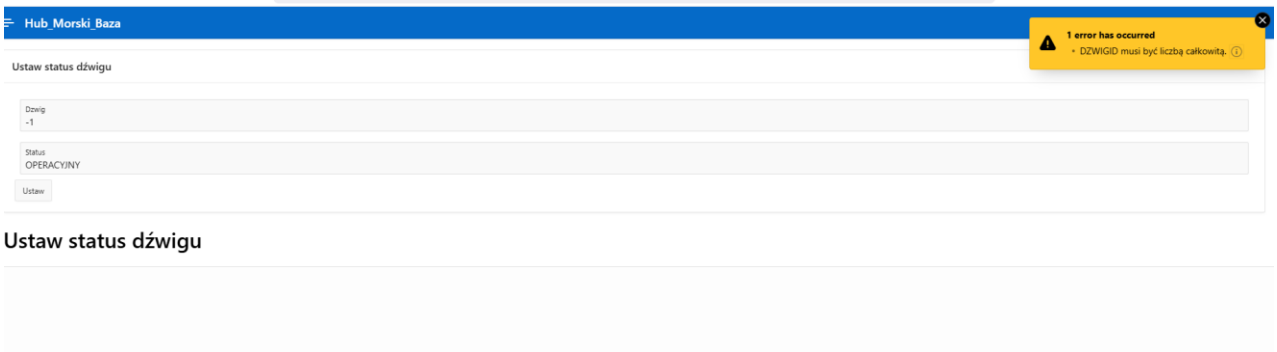
Błąd redefiniowany PRAGMA.

```
-- 1. DEFINICJA WYJĄTKU I PRAGMY
e_brak_dzwigu EXCEPTION;
PRAGMA EXCEPTION_INIT(e_brak_dzwigu, -20034);
```



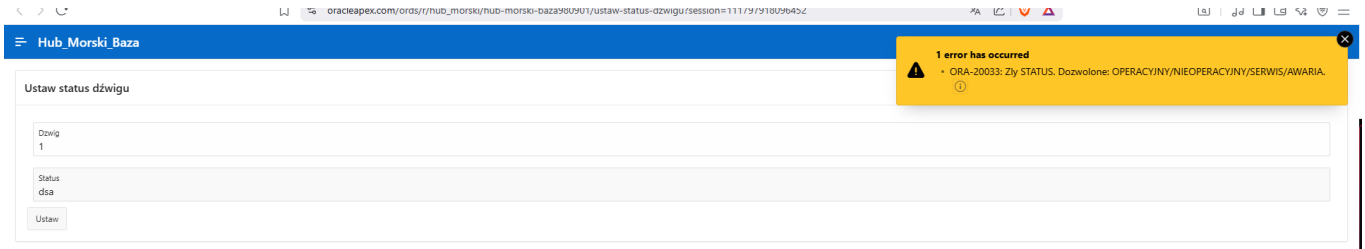
APEX Error

```
61
62  EXCEPTION
63    APEX Error
64    WHEN e_brak_dzwigu THEN
65      --
66      apex_error.add_error (
67          p_message          => 'Dźwig o podanym ID nie istnieje w bazie danych.',
68          p_display_location => apex_error.c_inline_in_notification
69      );
70
71    WHEN OTHERS THEN
72      --
73      apex_error.add_error (
74          p_message          => 'Wystąpił nieoczekiwany błąd: ' || SQLERRM,
75          p_display_location => apex_error.c_inline_in_notification
76      );
77  END;
78  /
```

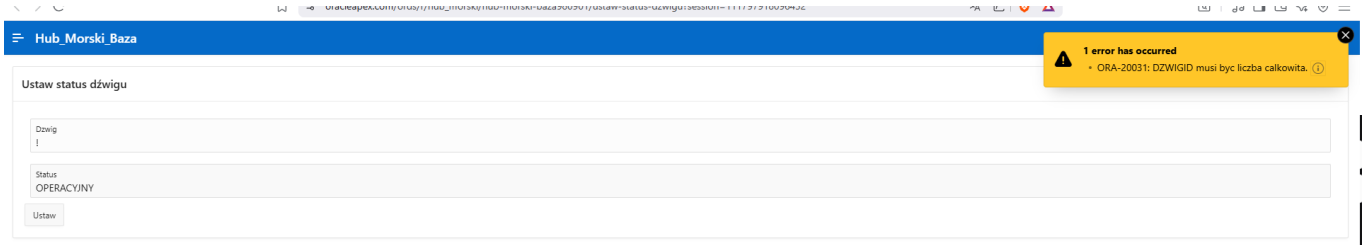


Raise aplication error

```
47
48  EXCEPTION
49    WHEN OTHERS THEN
50      IF SQLCODE BETWEEN -20999 AND -20000 THEN
51        RAISE;
52      END IF;
53      RAISE_APPLICATION_ERROR(-20099, 'Blad ustaw_status_dzwigu: ' || SQLERRM);
54  END;
55  /
```

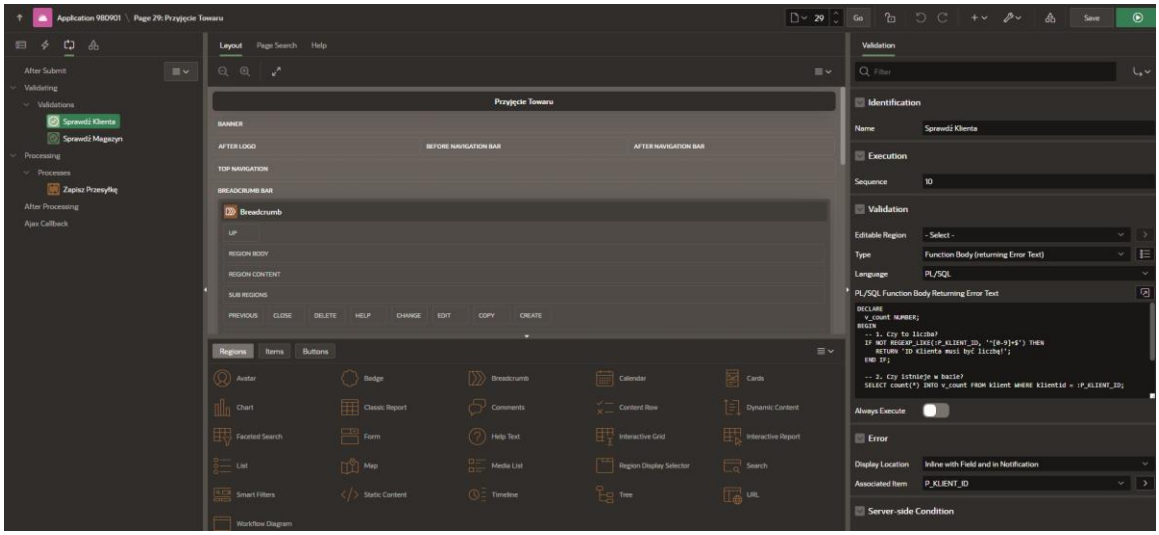


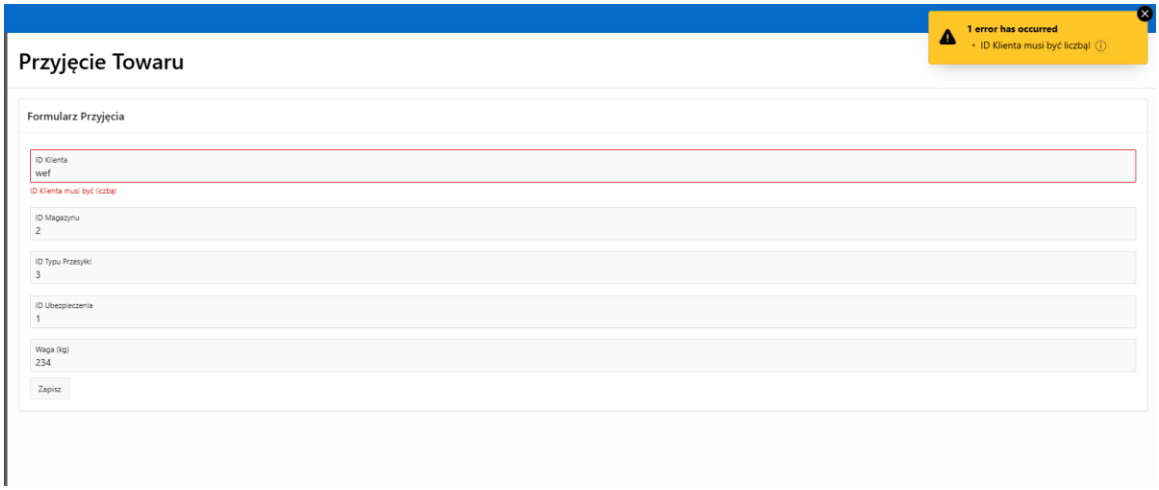
Ustaw status dźwigu



Ustaw status dźwigu

Walidacja w APEX.

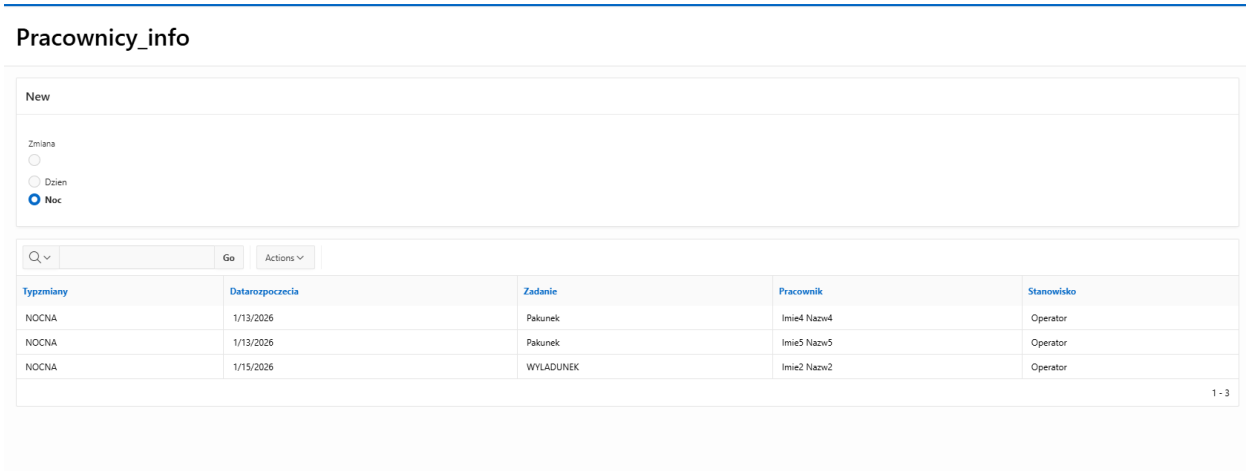


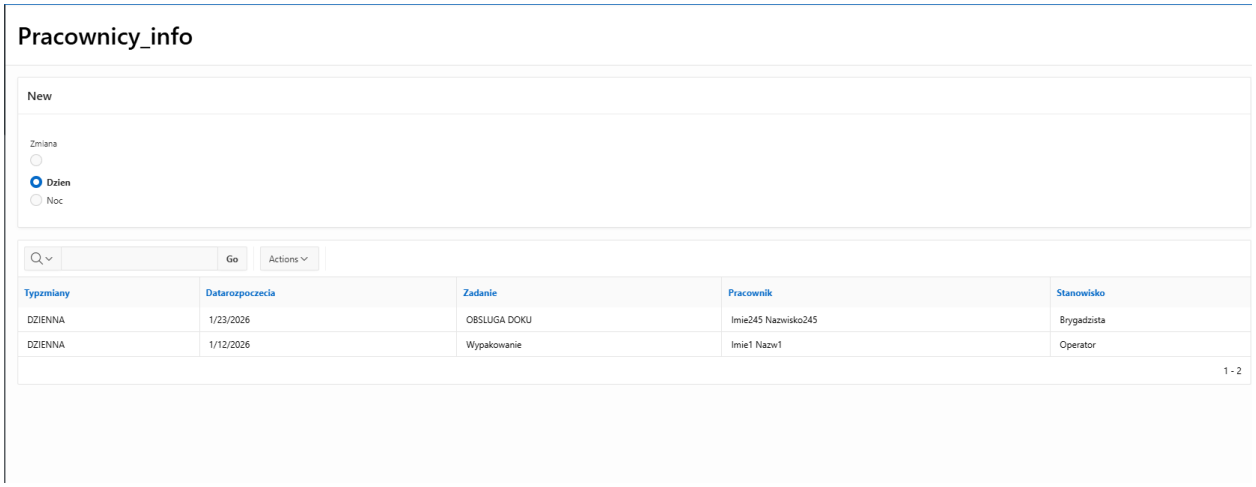


5. Strony APEX.

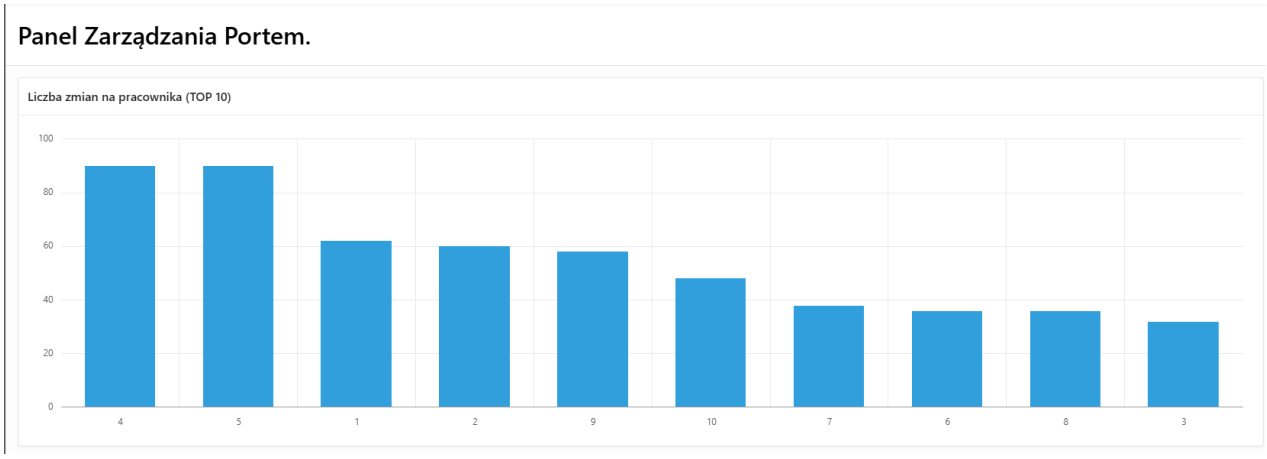
Kalendarz

Typy zmian podzial pracownikow dzienna/nocna

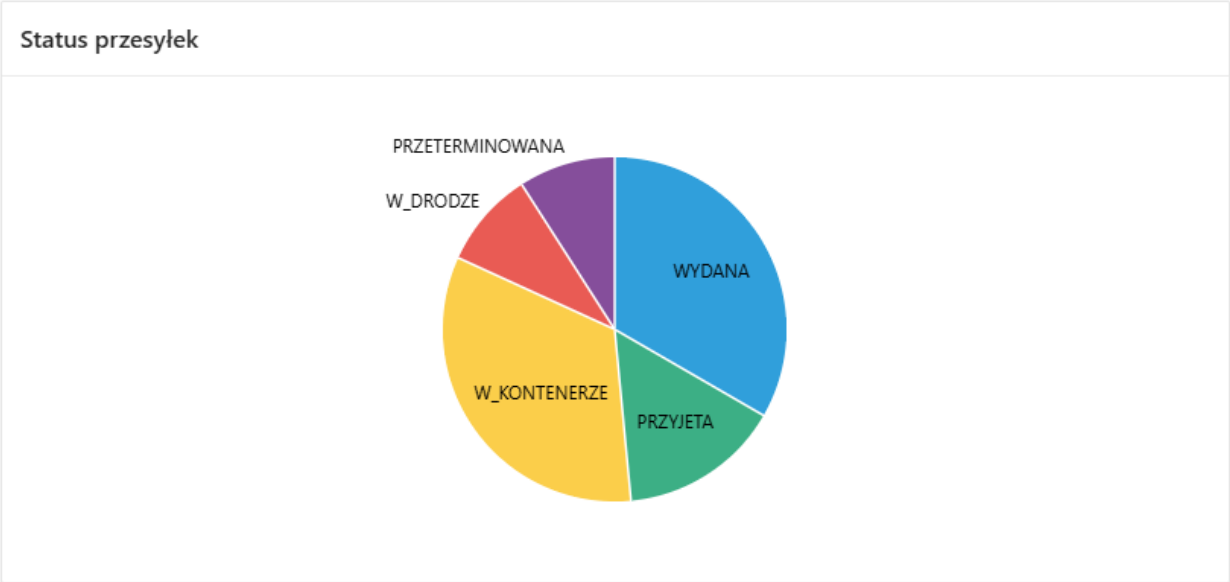


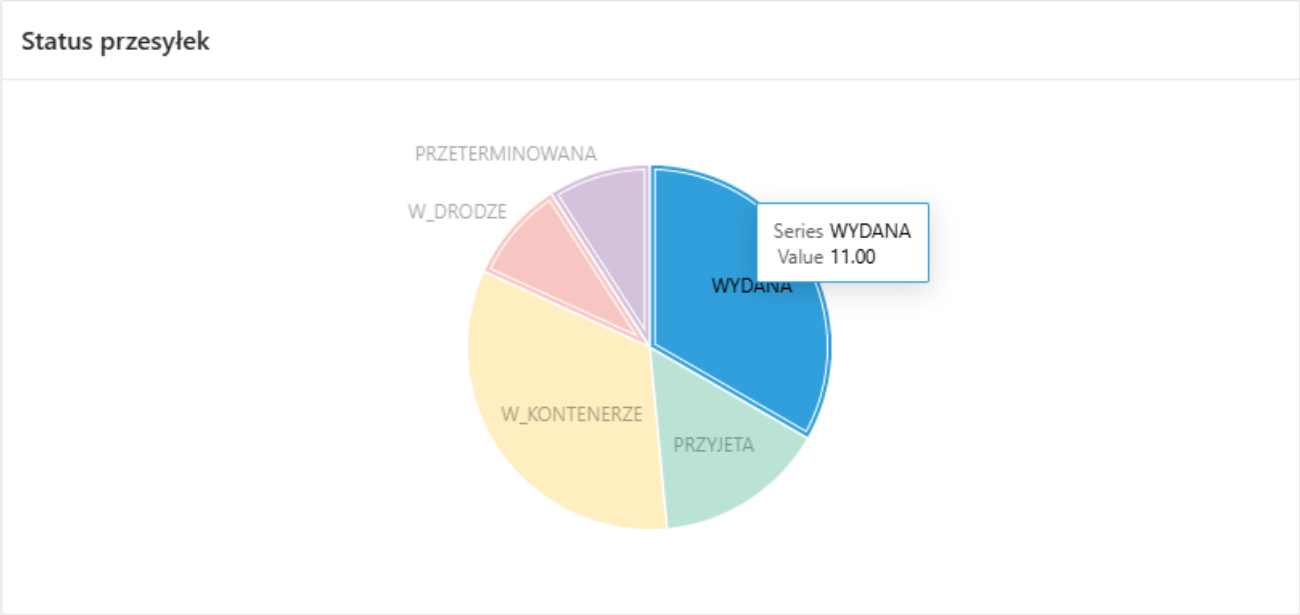


Wykres top 10 pracowników wedle ilości godzin (ID pracownika)

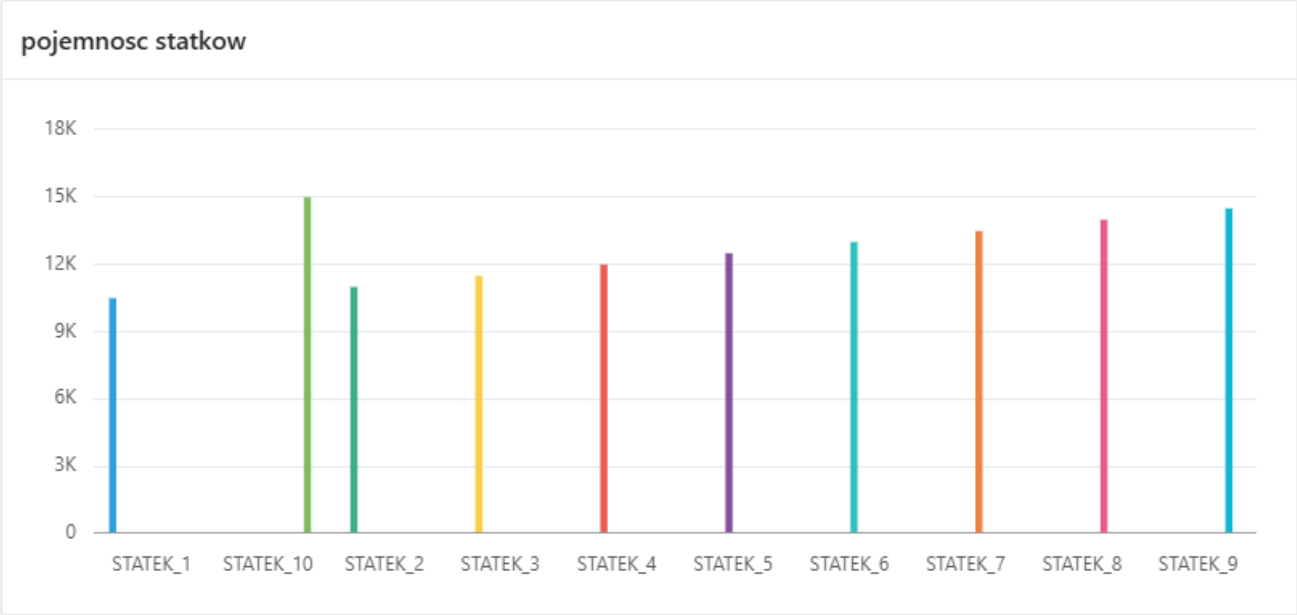


Wykres status przesyłek/ilość.



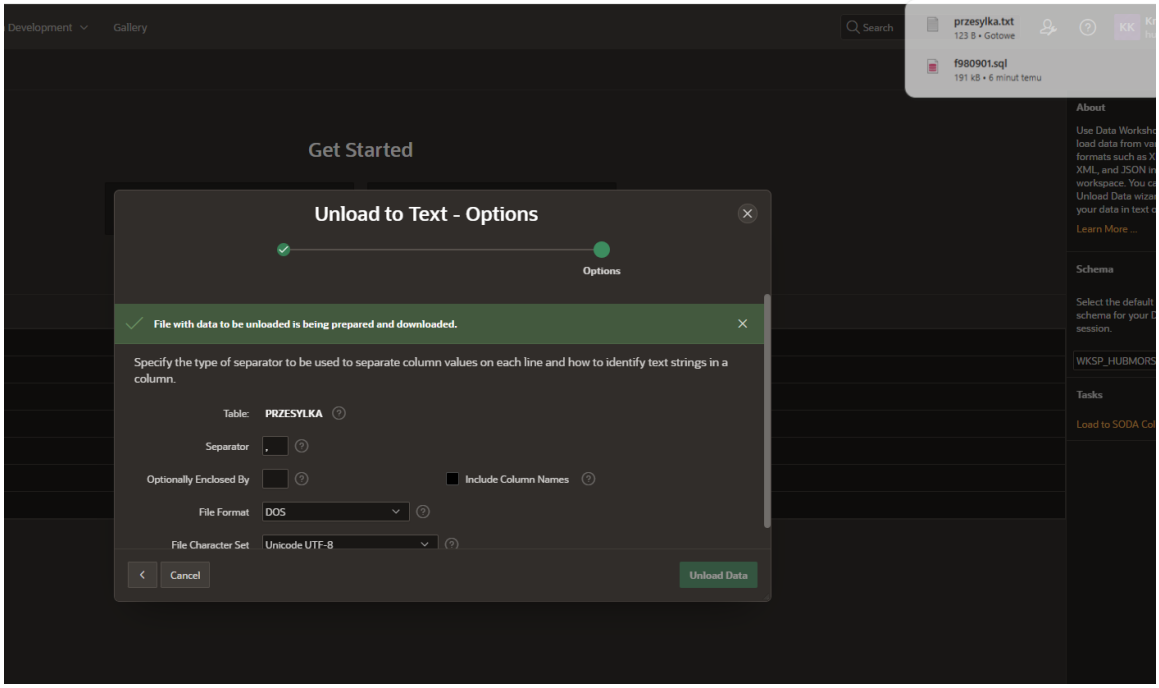


Wykres pojemność statków w tonach.

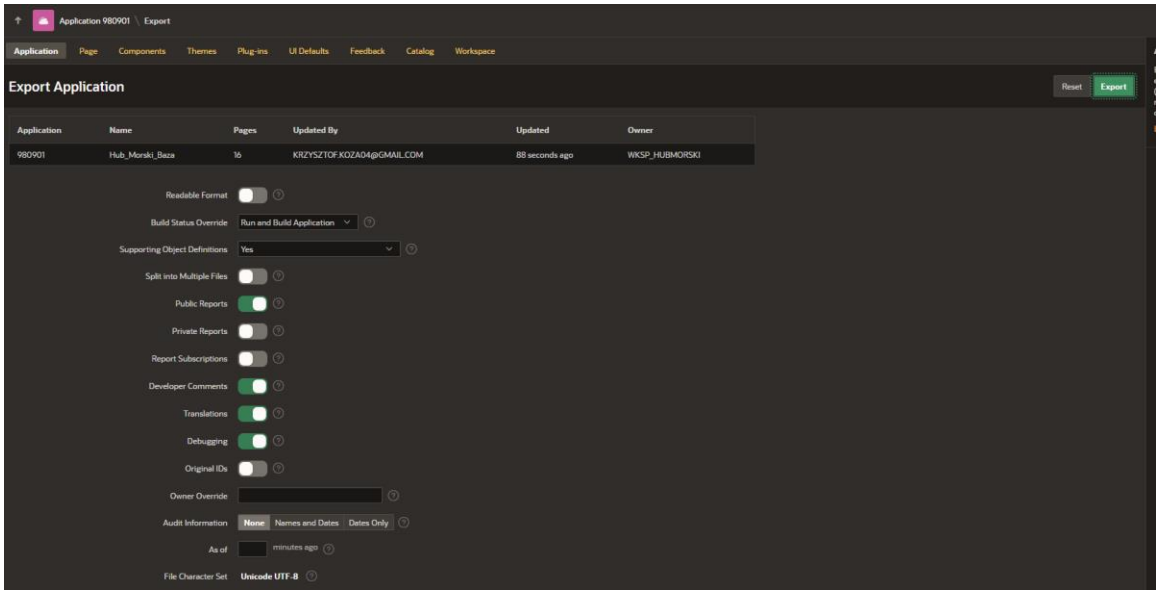


6. Eksport i import.

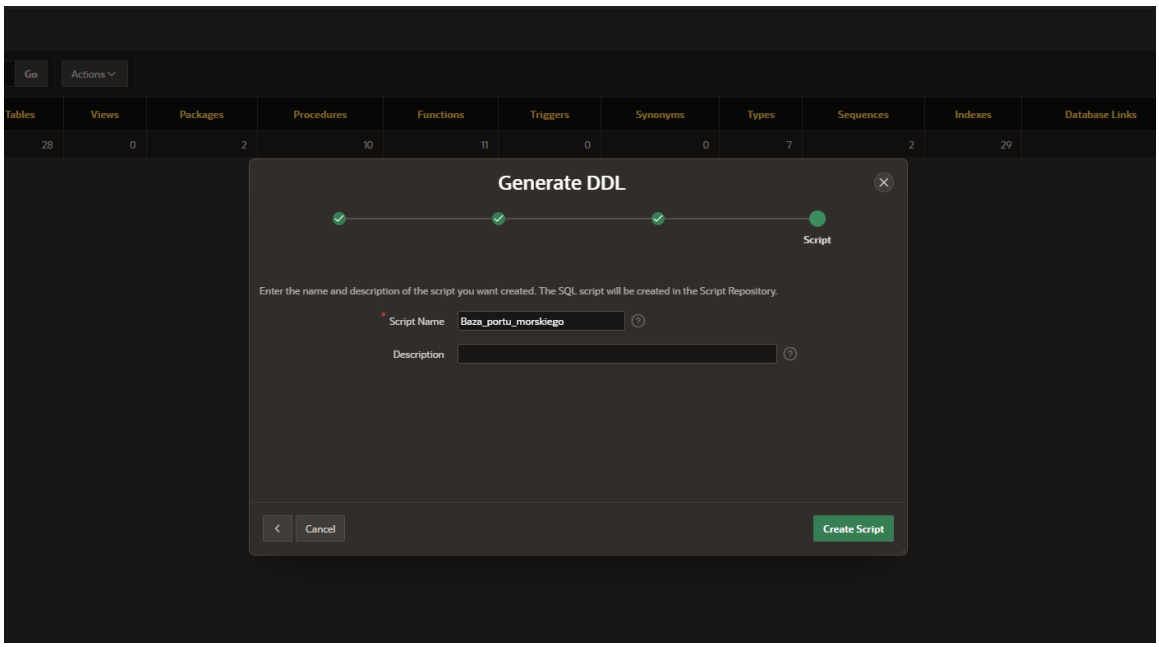
Eksport danych z tabel



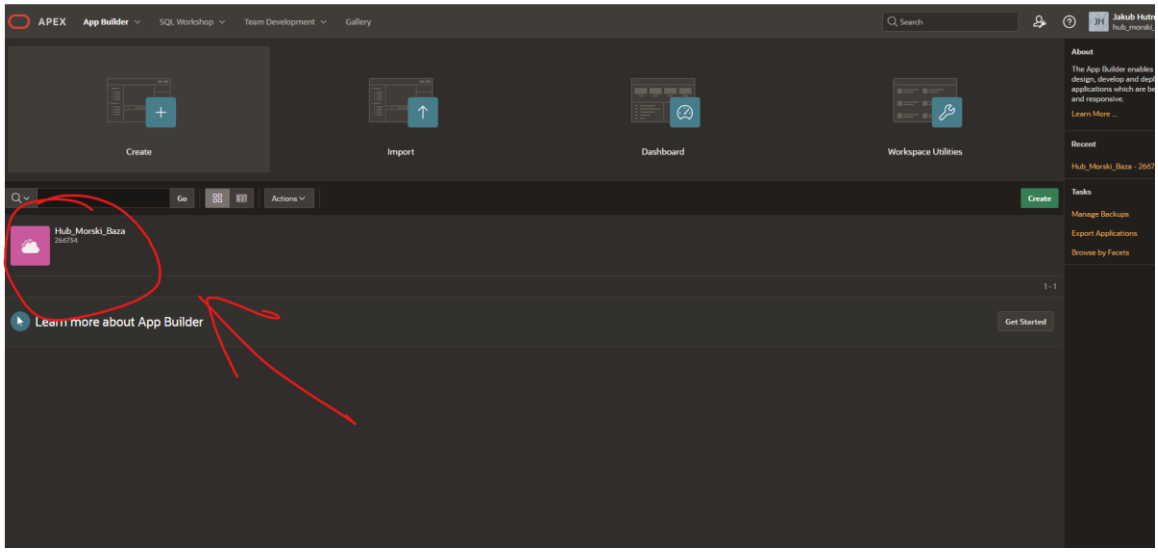
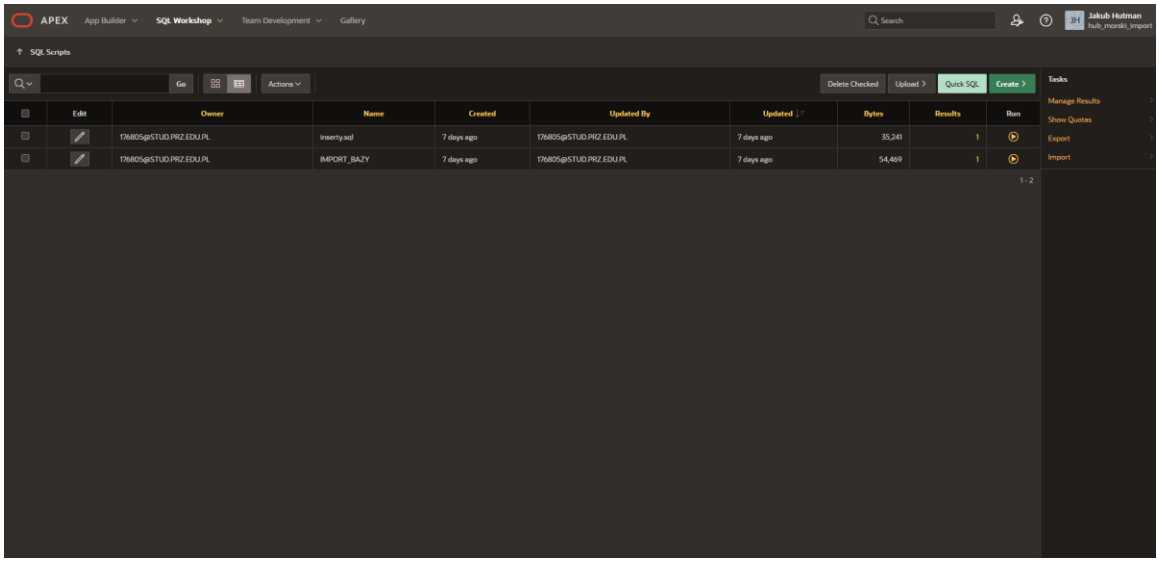
Eksport aplikacji



Eksport bazy danych



Import aplikacji do nowego workspace



Import bazy danych do nowego workspace